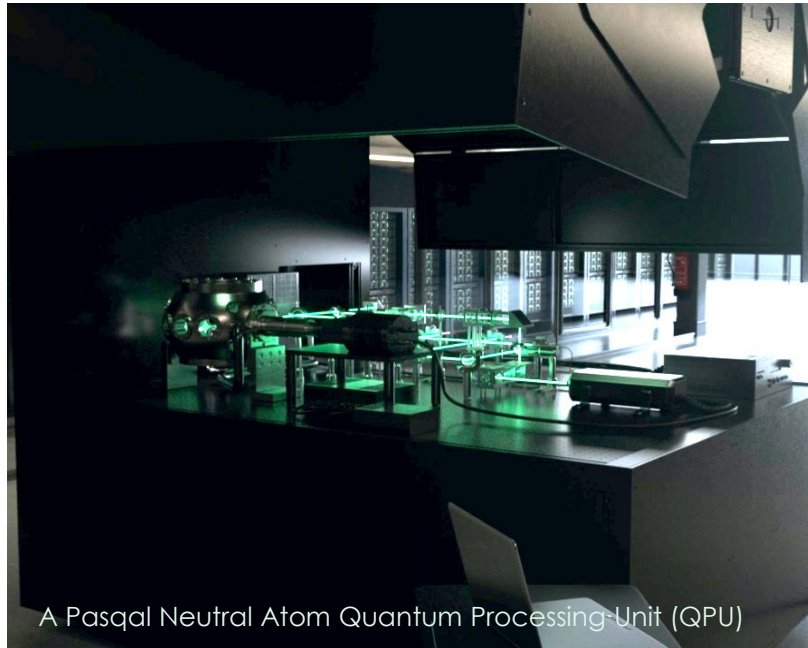


Using Logical Qubits to Solve Differential Equations

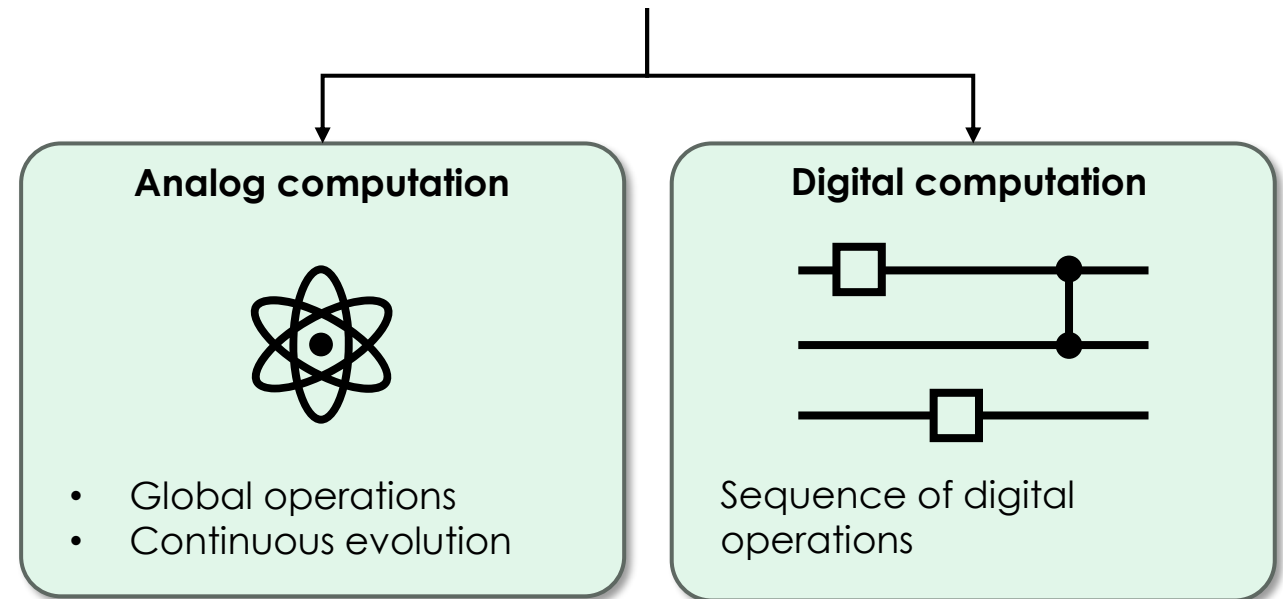
Evan Philip
2026 AlgoLab Summer School

Pasqal: a Full-Stack Quantum Computing Company



A Pasqal Neutral Atom Quantum Processing Unit (QPU)

Two distinct computation approaches:



Today's focus

- I work on algorithms, primarily in Quantum Machine Learning.
- Pasqal has been improving the digital capabilities of its neutral atom quantum processors.
- The goal was to do some co-designing: run a small algorithm **with and without error correction** and compare the performance.
We wanted to compare on algorithm-level performance (as opposed to metrics like gate fidelity).
- I would like to share my perspective of this journey (non-expert in hardware/error correction).

Introduction



Goal of the Work

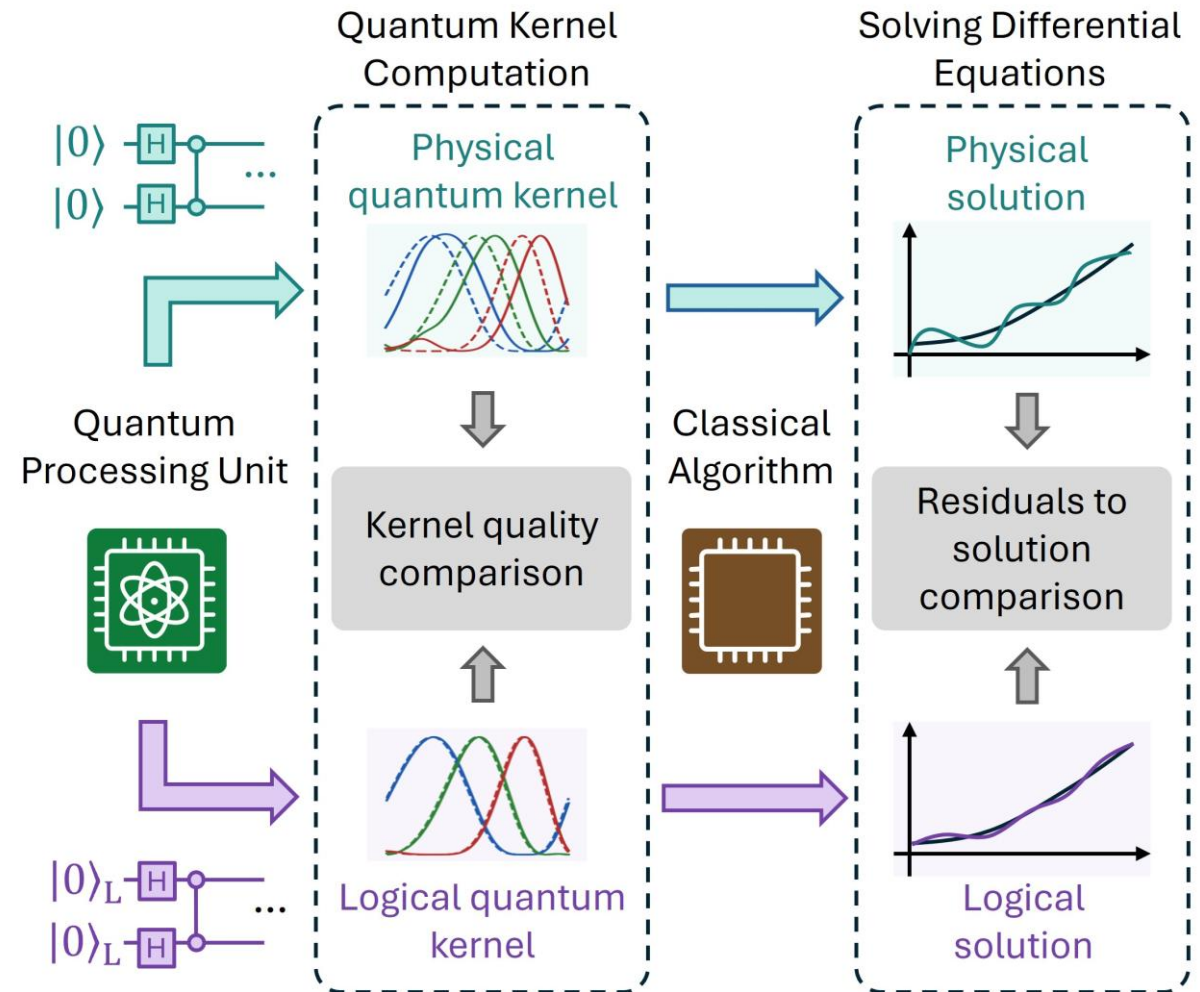
Goal:

Test the performances of our logical qubits by comparing them to our physical qubits on an **end-to-end** application

- We decided to solve differential equations.
- We opted to do so using kernel mixed-model regression [1].

Choice:

Solving differential equation using a quantum kernel: interesting, industrially-relevant application



Key Points of the Method

Key points of the method:

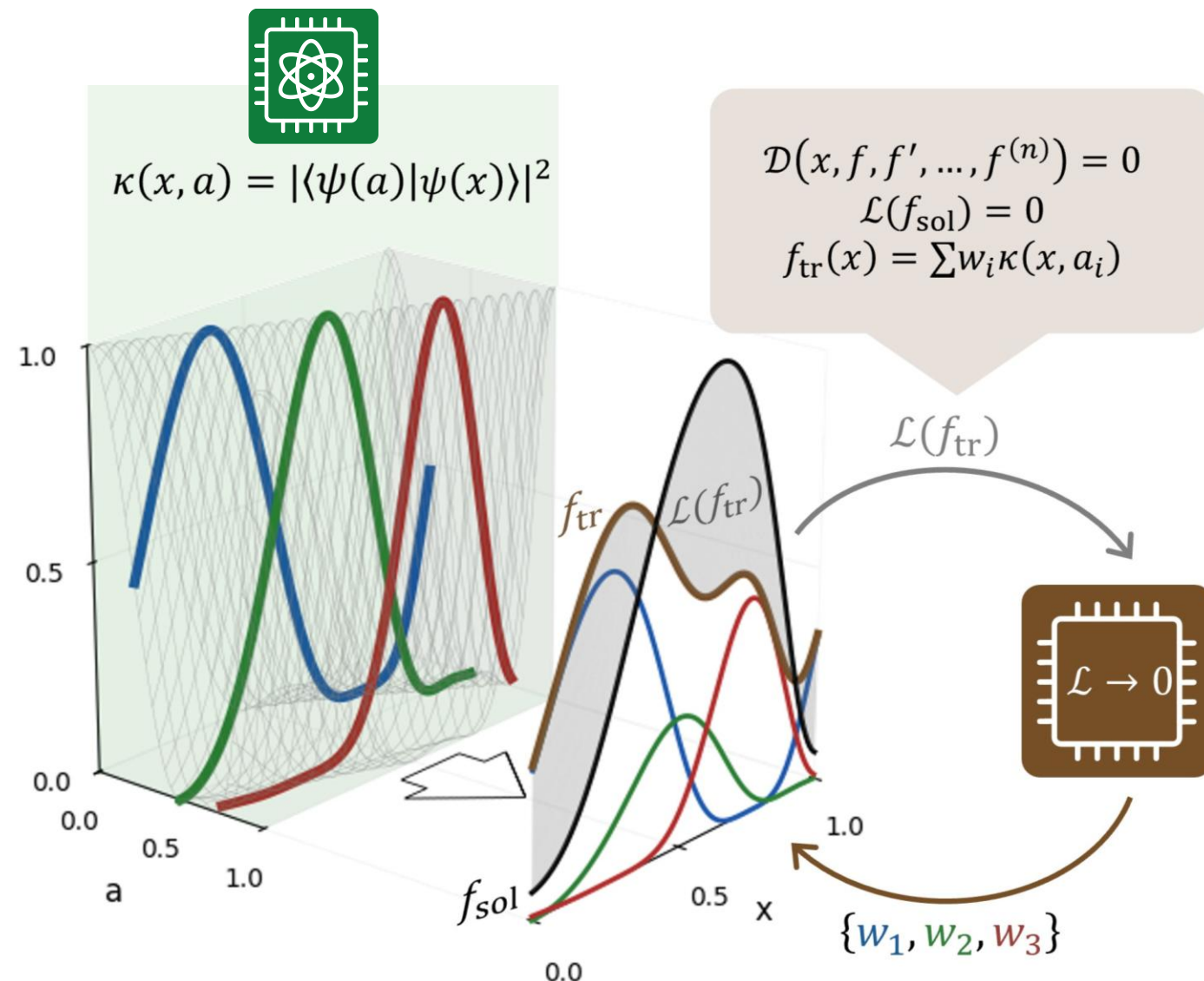
- The quantum kernel provides a basis for a trial function

$$f_{\text{tr}}(x) = \sum w_i \kappa(x, a_i) + c$$

- Minimize classically:

$$\mathcal{L}(f_{\text{tr}}) \rightarrow 0$$

- The quantum kernel here is a fixed resource (no iterative feedback) that is a part of an otherwise classical algorithm.

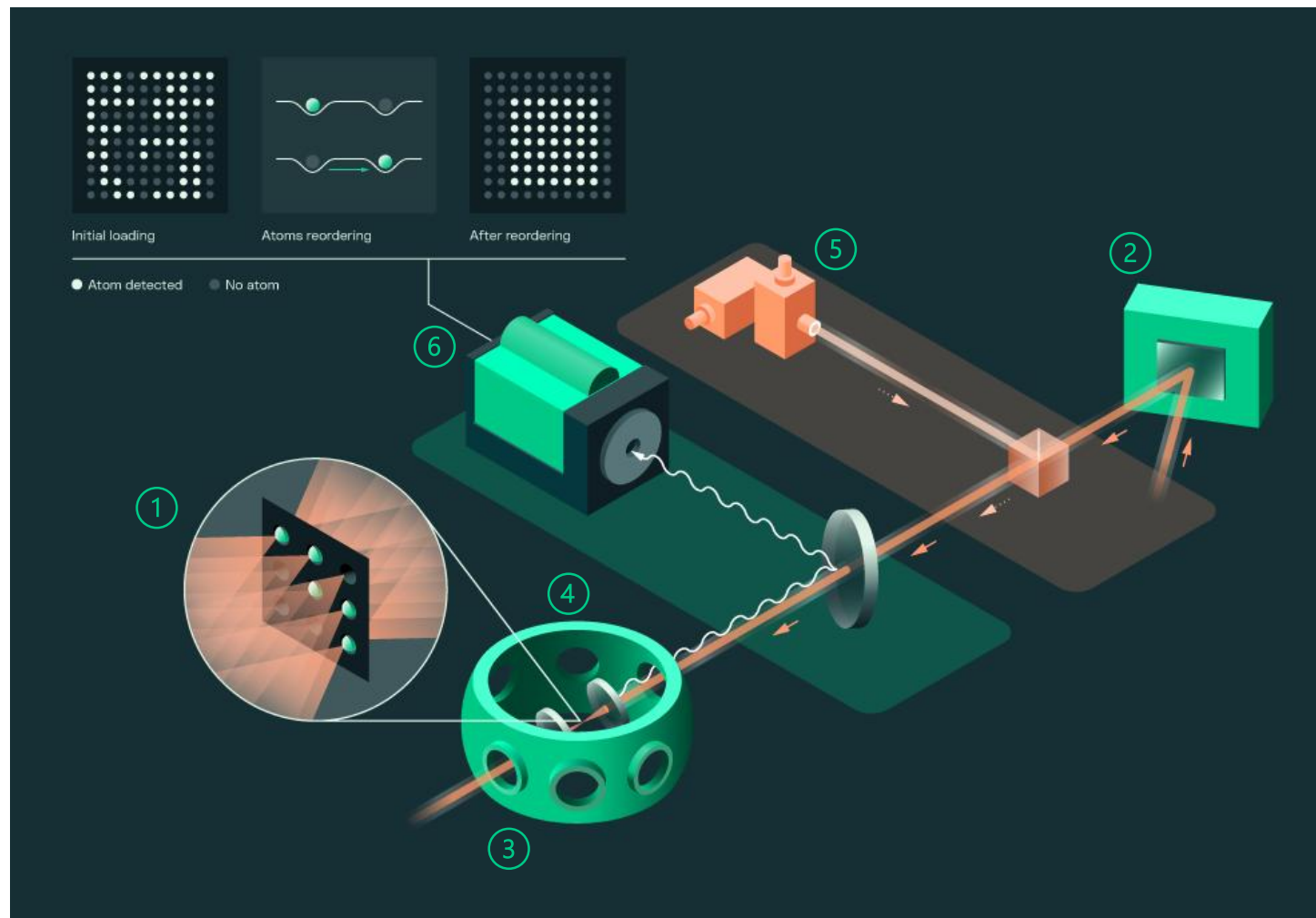


| Neutral Atom QPU



How does a neutral atom quantum processor work?

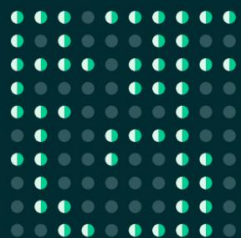
The core components inside a Pasqal QPU



1. Atoms are trapped by a laser
2. Multiple traps are created by splitting the laser with a Spatial Light Modulator (SLM)
3. Vacuum chamber to keep atoms from any disturbances
4. Two high numerical aperture lenses to focus the laser spots
5. Moving atoms is enabled by Acousto-Optical Deflectors (AODs)
6. Readout of the atoms with a fluorescent light detecting camera

How does a neutral atom quantum processor work?

Encoding and processing quantum bits, or qubits



Atom loading

A laser acts like an optical tweezer, trapping atoms

Multiple traps are generated by splitting the laser beam

Some of the traps will contain an atom due to stochastics



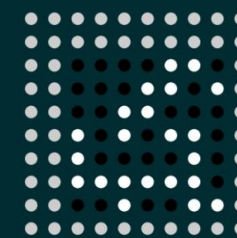
Register preparation

Atoms can be moved to create custom grids, forming the quantum register



Quantum processing

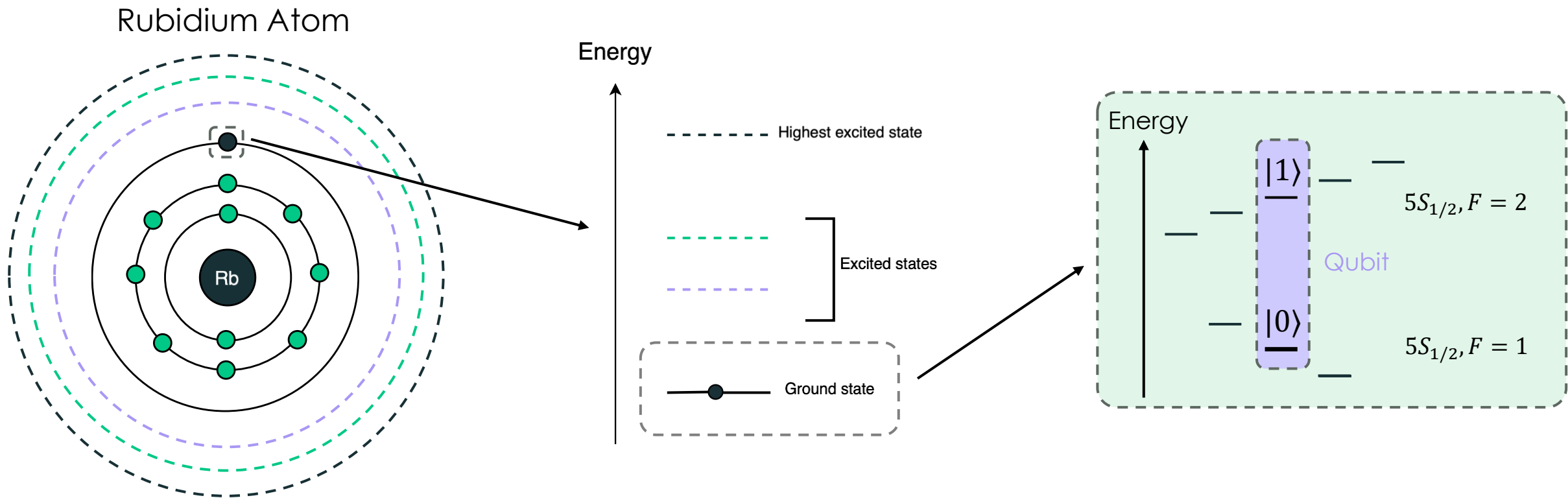
Manipulating the array of atoms with other lasers allows for quantum calculations by switching atoms between energy states



Results read-out

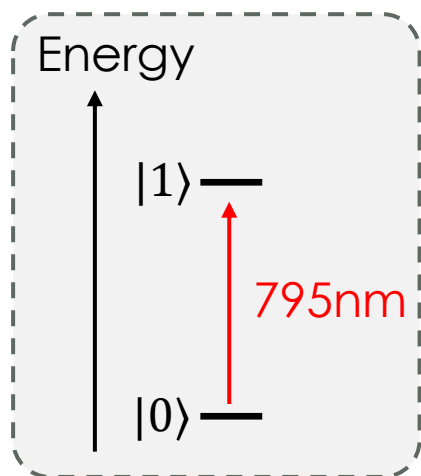
Final results are read by imaging the array: excited atoms (ones) are dark, ground-state atoms (zeros) are bright

Atomic States and Qubit Encoding



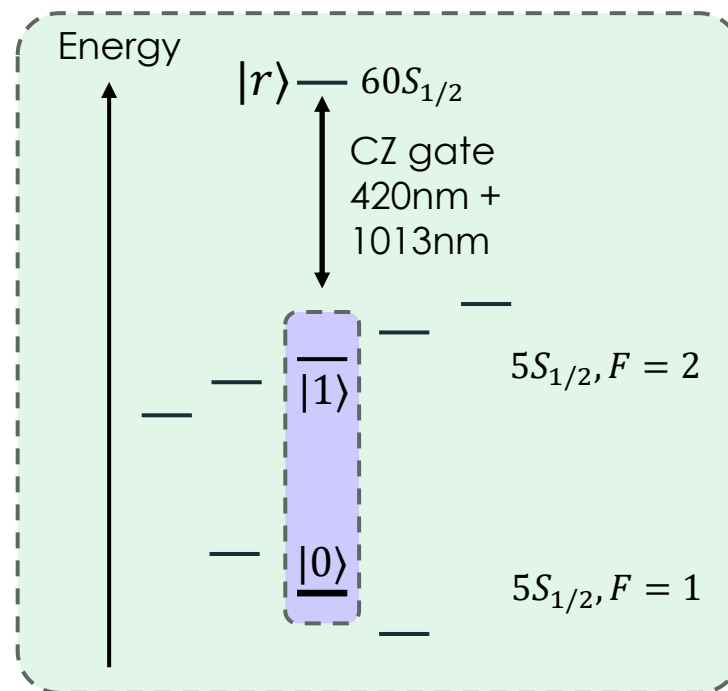
Atomic States and Two-Qubit Gates

Single qubit:

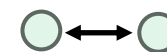


Microwave-regime transition performed via a laser thanks to Raman transitions

Two qubits:



$$R < R_b$$



CZ gate

$|00\rangle \rightarrow |00\rangle$
 $|01\rangle \rightarrow |01\rangle$
 $|10\rangle \rightarrow |10\rangle$
 $|11\rangle \rightarrow -|11\rangle$

$$R > R_b$$



Identity

$|00\rangle \rightarrow |00\rangle$
 $|01\rangle \rightarrow |01\rangle$
 $|10\rangle \rightarrow |10\rangle$
 $|11\rangle \rightarrow |11\rangle$

Entangling operation is accomplished utilizing **strong interaction** between atoms if simultaneously in **Rydberg state** while in **proximity** ($1/R^6$).

Universal computing by combining arbitrary single-qubit gates and CZ gates

How do we handle more qubits?

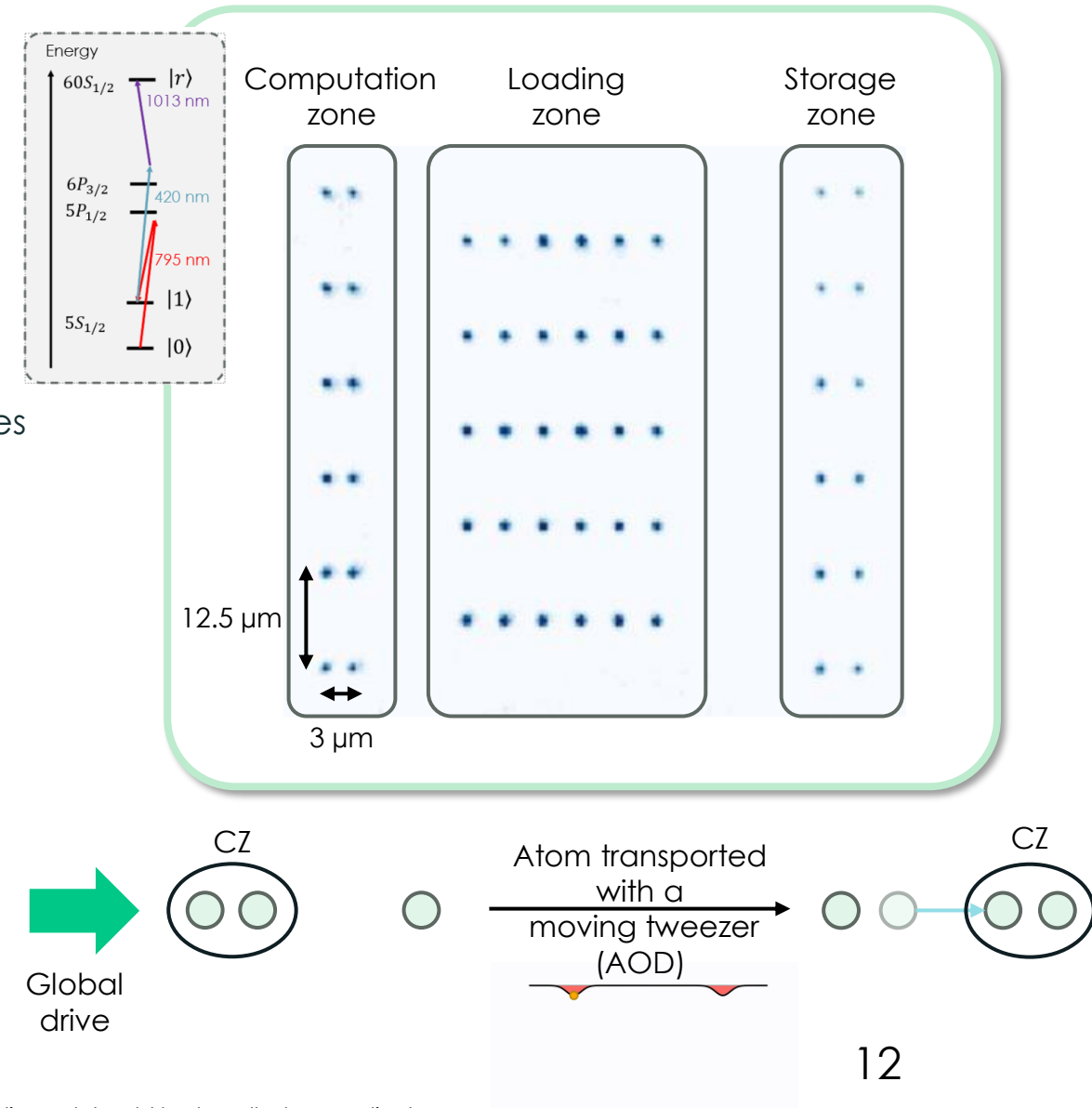
Architecture and local gates:

Register space (SLM) divided into **3 zones** [1-3]:

- Loading zone
- Computation zone: global illumination for gates
- Storage zone: atoms that are in this zone are insensitive to gates

(Readout done on the whole register)

- Arbitrary local single-qubit gates: **combination of global illumination** of the computation zone with **atom movement** between zones
- Two-qubit gate (CZ) locality: **set by interatomic distance** (Rydberg blockade) [4]
- State-preservation during movement: Compensation of the accumulated dephasing via spin-echo **dynamical decoupling** [2-3,5]



- [1] J. Beugnon *et al.* (2007)
- [2] D. Bluvstein *et al.* (2022)
- [3] D. Bluvstein *et al.* (2024)
- [4] H. Levine *et al.* (2019)
- [5] H.J. Manetsch *et al.* (2025)

| Error Detecting Code



Error Detecting Code

[[4,2,2]] error detecting code (little Shor code)

Notation $[[n, k, d]]$

- $n = 4$: physical data qubits
- $k = 2$: encoded logical qubits
- $d = 2$: code distance (error detection)

Distance 2 : one error is detectable but not correctable

Stabilizers

$$S_X = X \otimes X \otimes X \otimes X$$

$$S_Z = Z \otimes Z \otimes Z \otimes Z$$

Valid codewords are in the +1 eigenspace of the stabilizers

Logical codewords

$$|00\rangle_L \rightarrow |0000\rangle + |1111\rangle,$$

$$|01\rangle_L \rightarrow |1100\rangle + |0011\rangle$$

$$|10\rangle_L \rightarrow |1010\rangle + |0101\rangle,$$

$$|11\rangle_L \rightarrow |0110\rangle + |1001\rangle$$

Lot of implementations: Dam et al. (2024) (Microsoft), Yamamoto et al. (2024) (Quantinuum), Bedalov et al. (2024) (Infleqtion), Reichardt et al. (2024) (Atom Computing), Gupta et al. (2024) (IBM)

Transversal gates set

Operation	Unencoded	FT Encoded
$X(q_0)$		
$Z(q_0)$		
$X(q_1)$		
$Z(q_1)$		
$CZ(q_0, q_1)$		
$CX(q_0, q_1)$		
$CX(q_1, q_0)$		
$HH(q_0, q_1)$		

Implementing the Error Detecting Code on Hardware

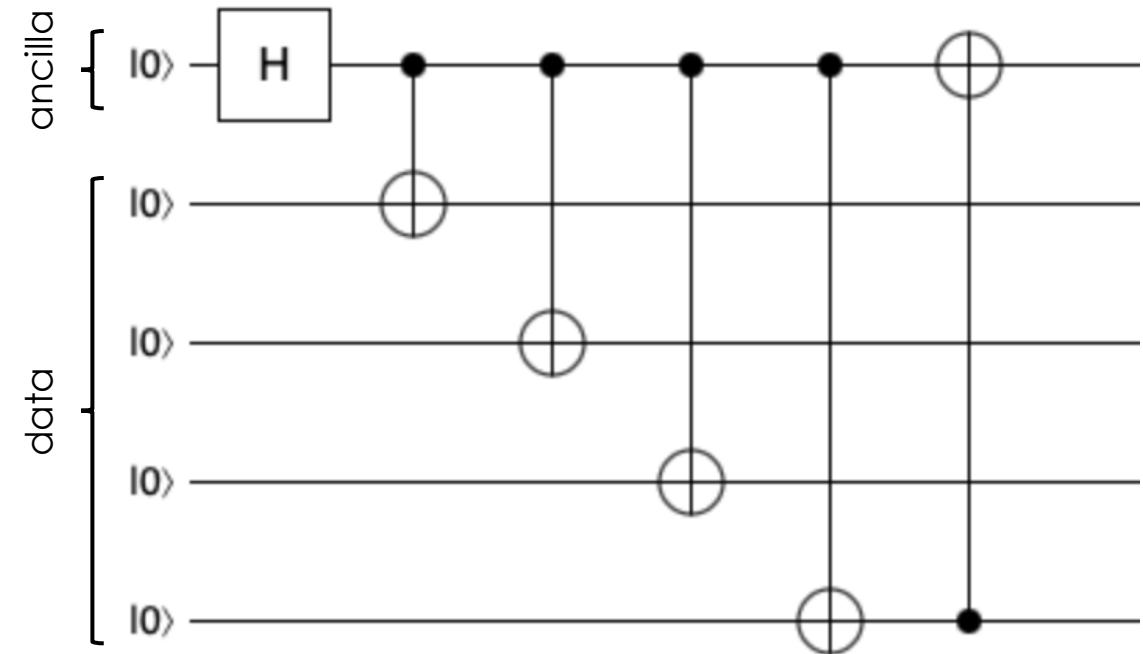
Target state – the initial state :

$$|00\rangle_L = \frac{1}{\sqrt{2}} (|0000\rangle + |1111\rangle)$$

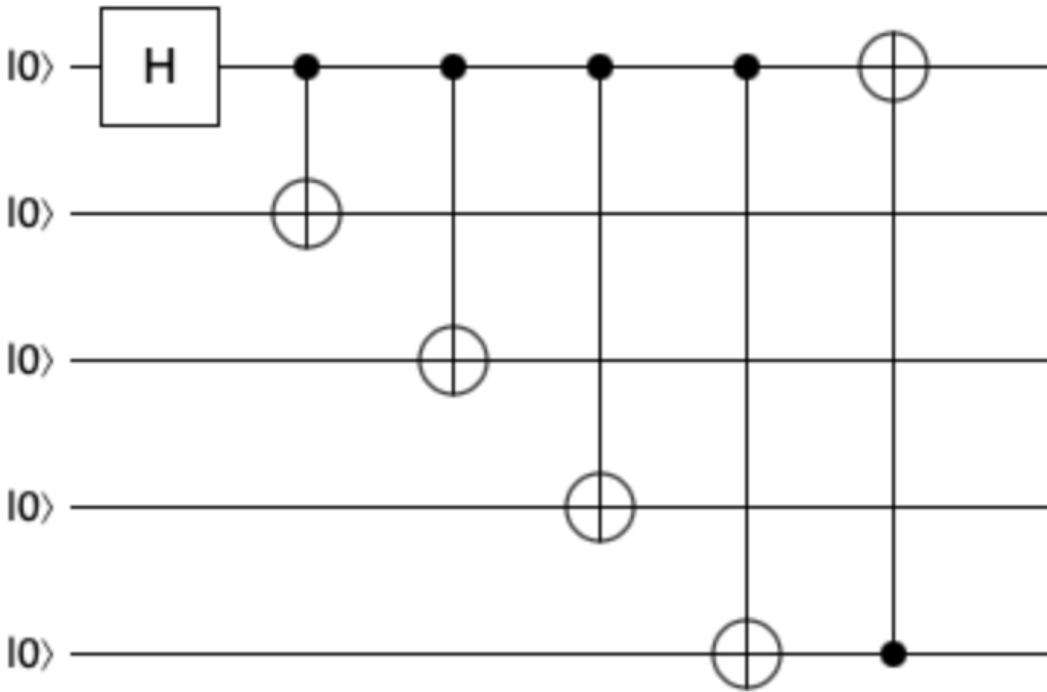
4 data qubits + one ancilla qubit for post-selection

Errors are detected via:

- The final state of the **ancilla qubit**
- The **parity** of the final state (codespace)



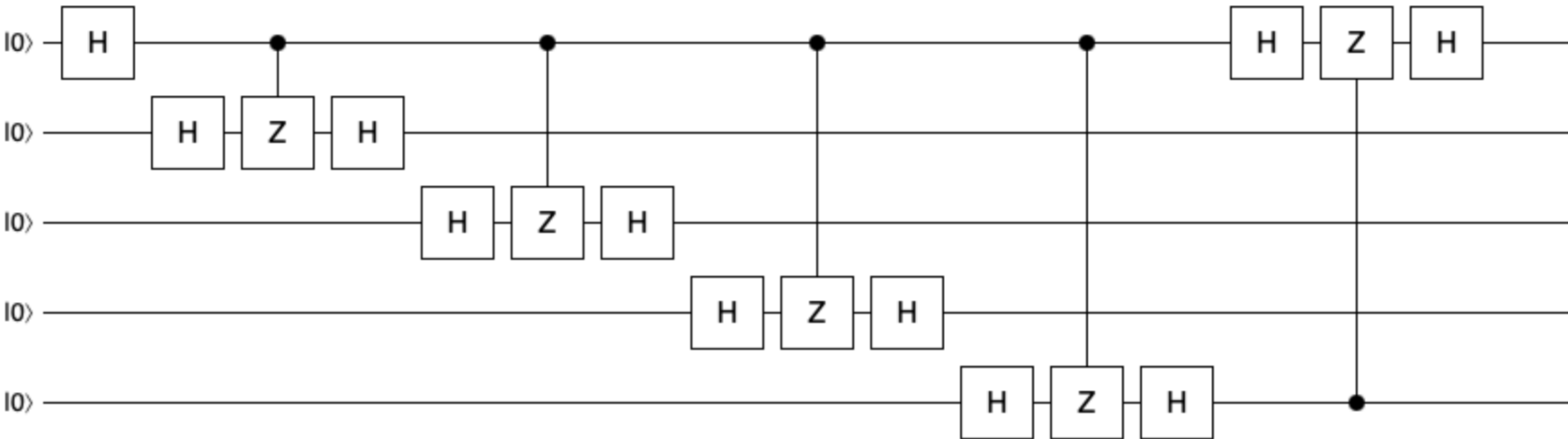
Quantum circuit, from theory to hardware



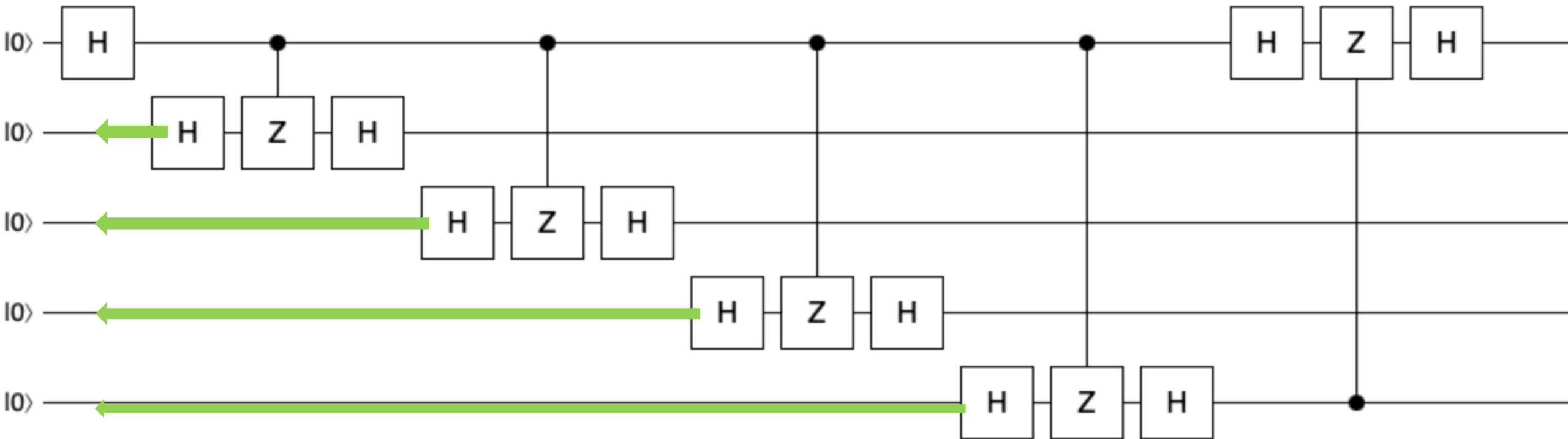
2-qubit gates $\rightarrow$ CZ

(locality via
interatomic distance)

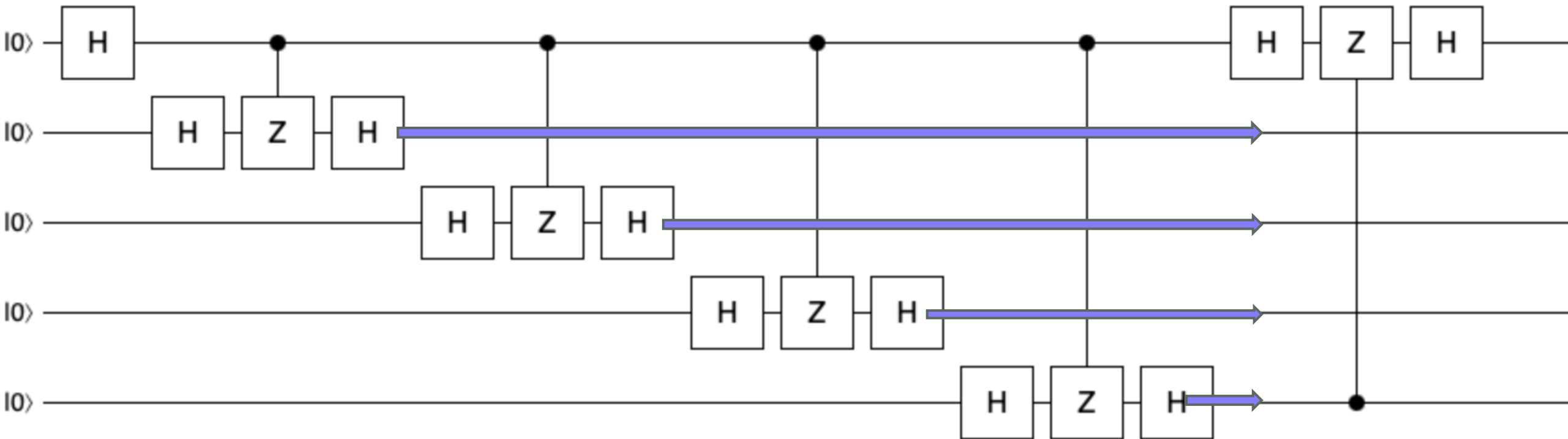
Quantum circuit, from theory to hardware



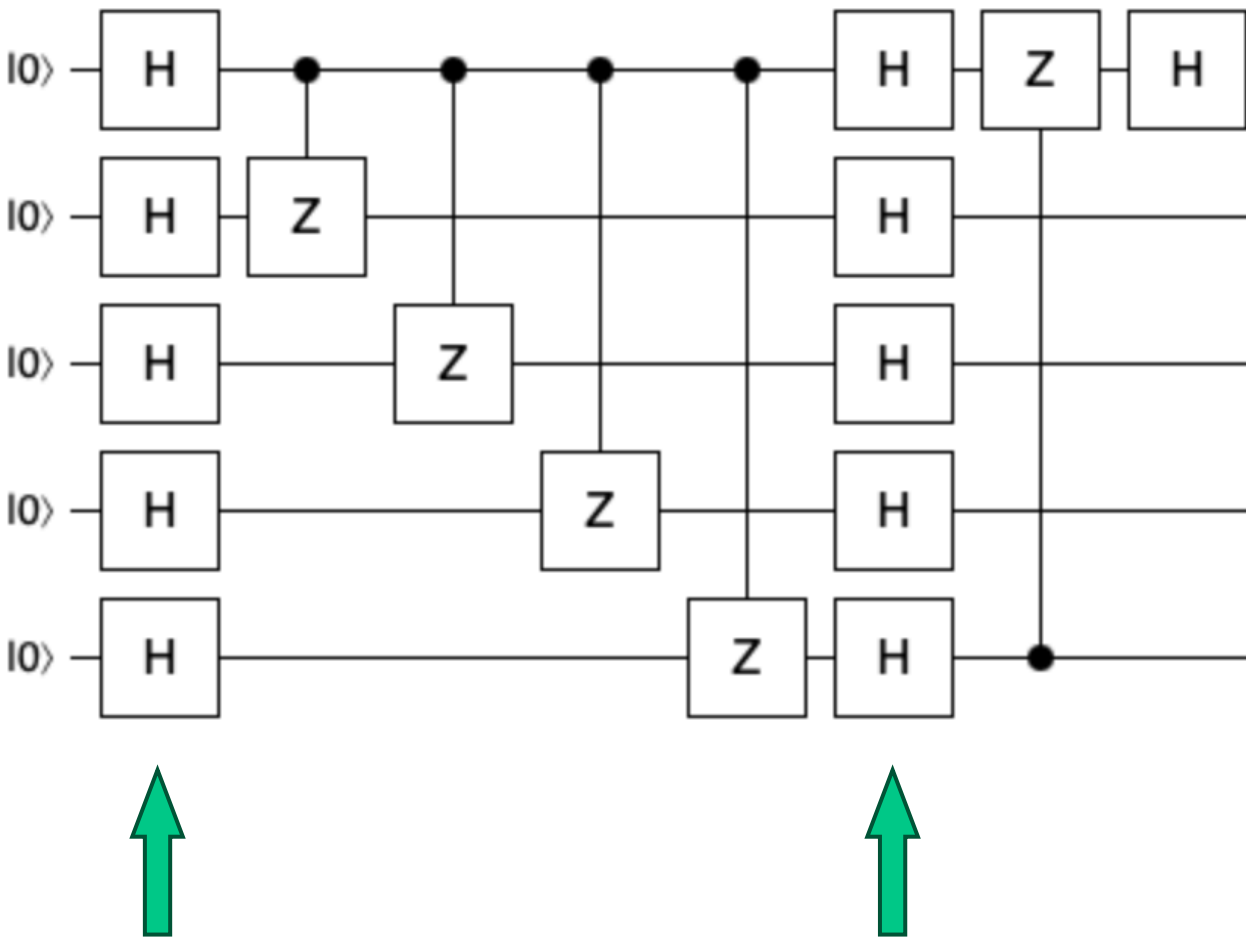
Quantum circuit, from theory to hardware



Quantum circuit, from theory to hardware

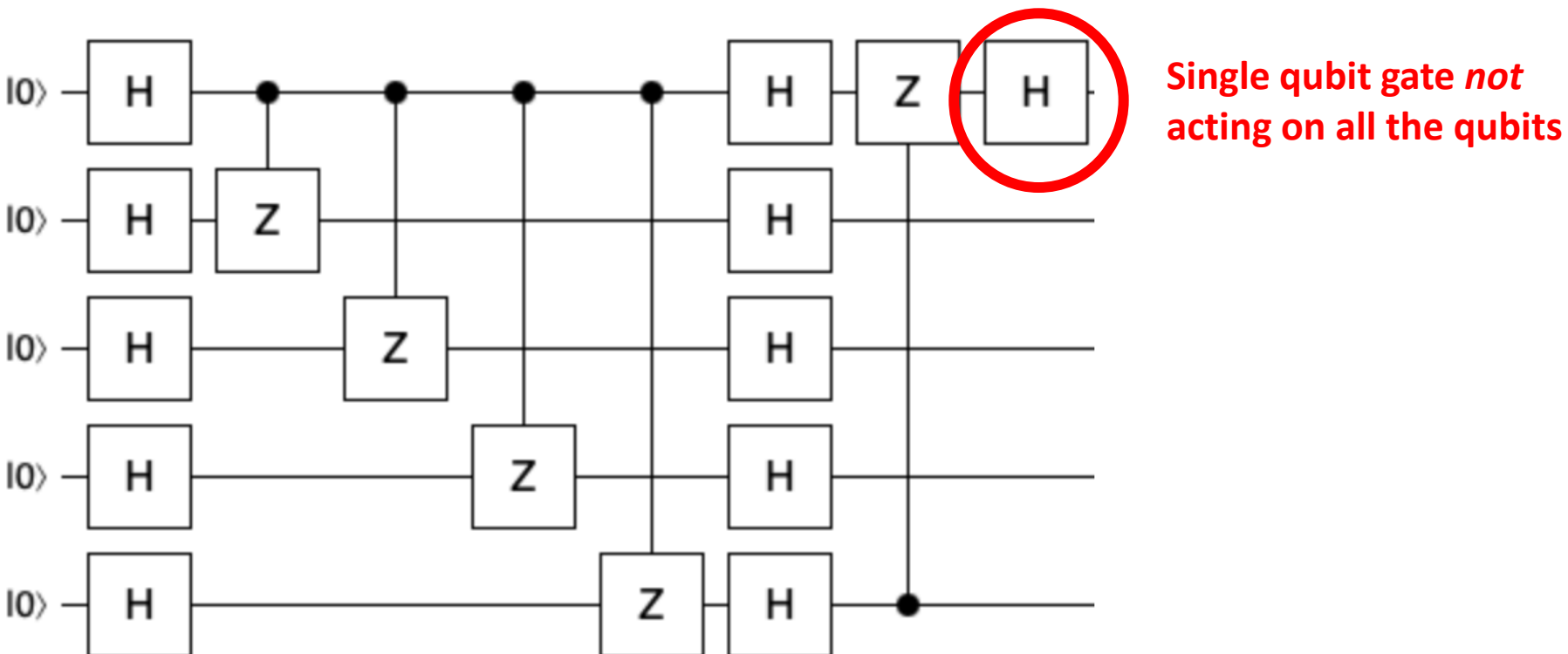


Quantum circuit, from theory to hardware

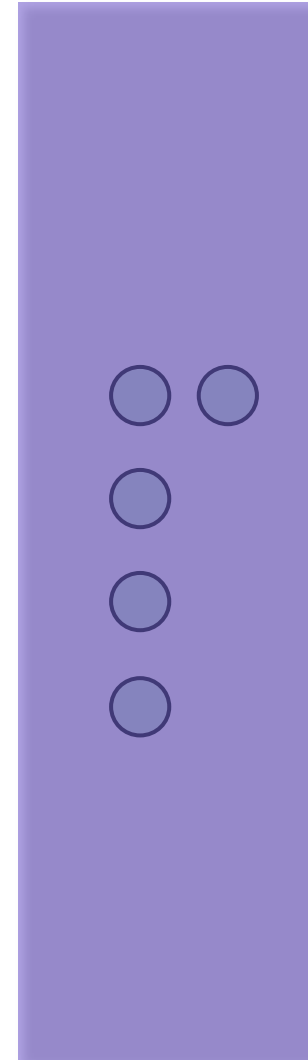
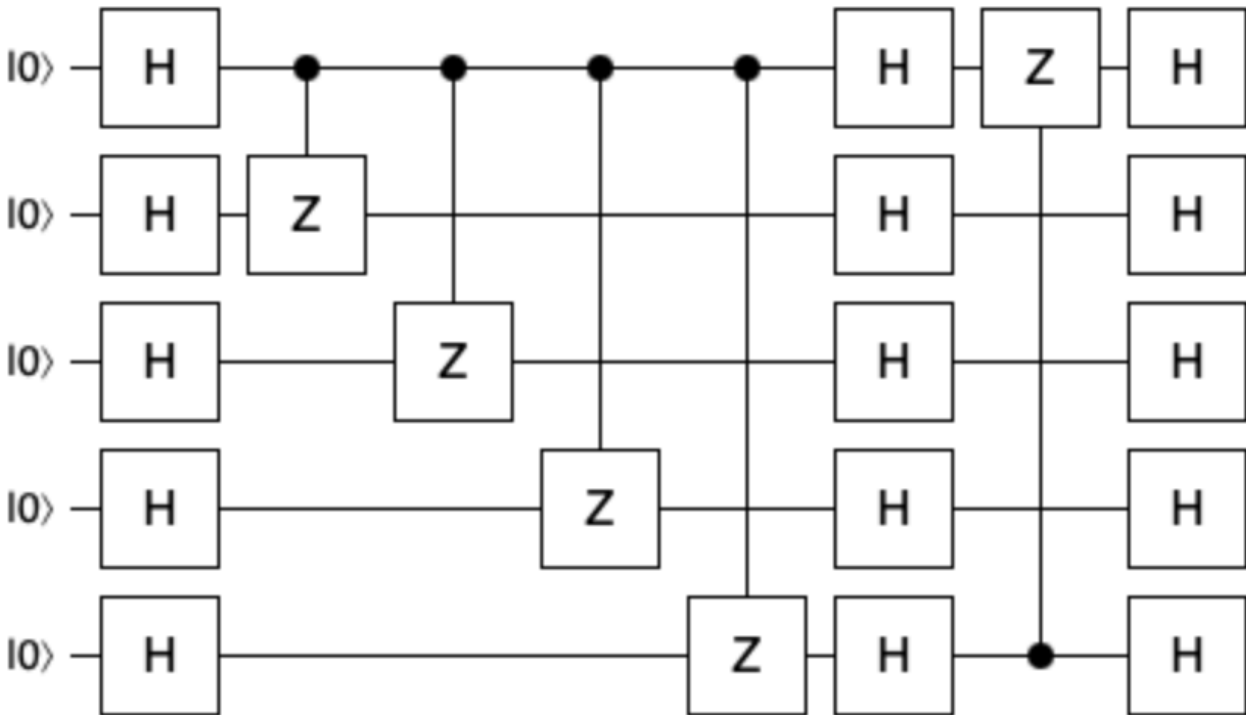


Single qubit gates via
global illumination of
the computational
zone

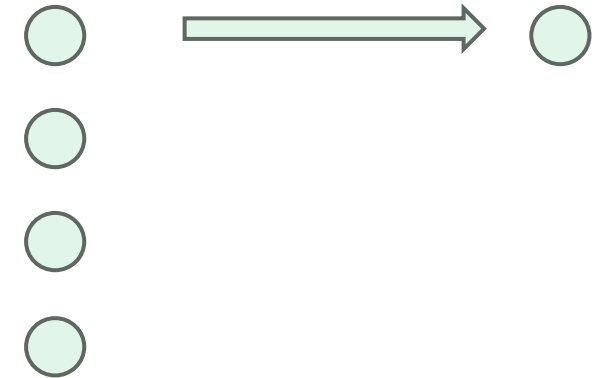
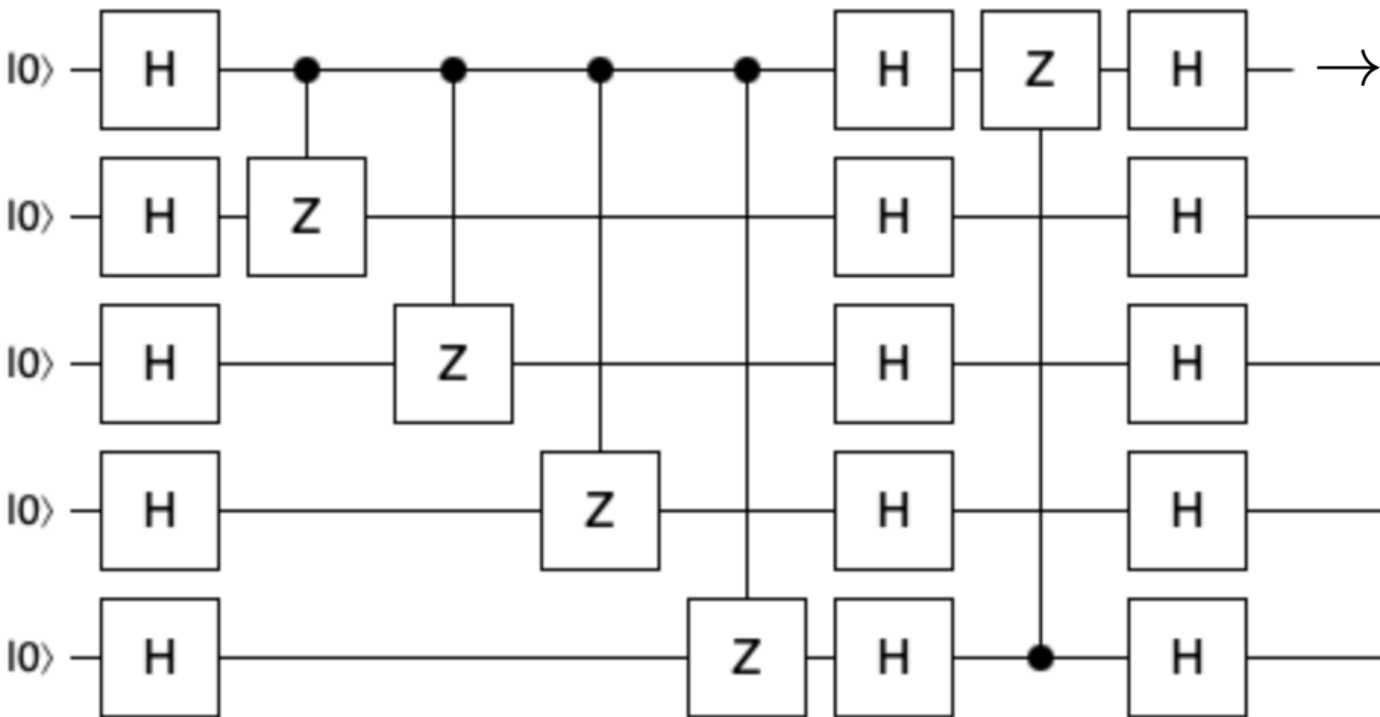
Quantum circuit, from theory to hardware



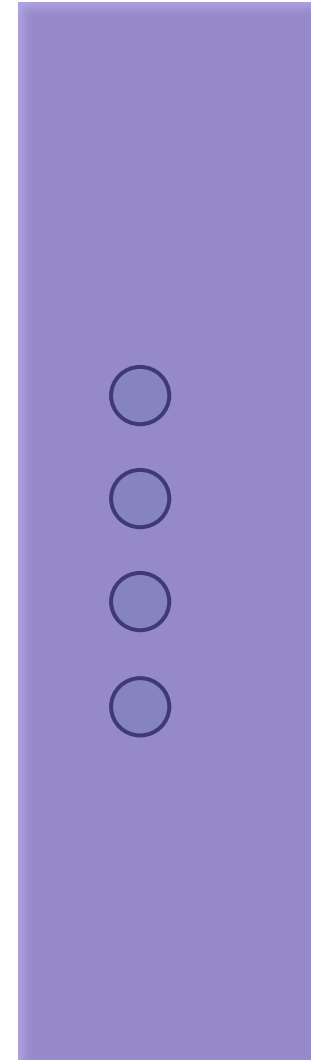
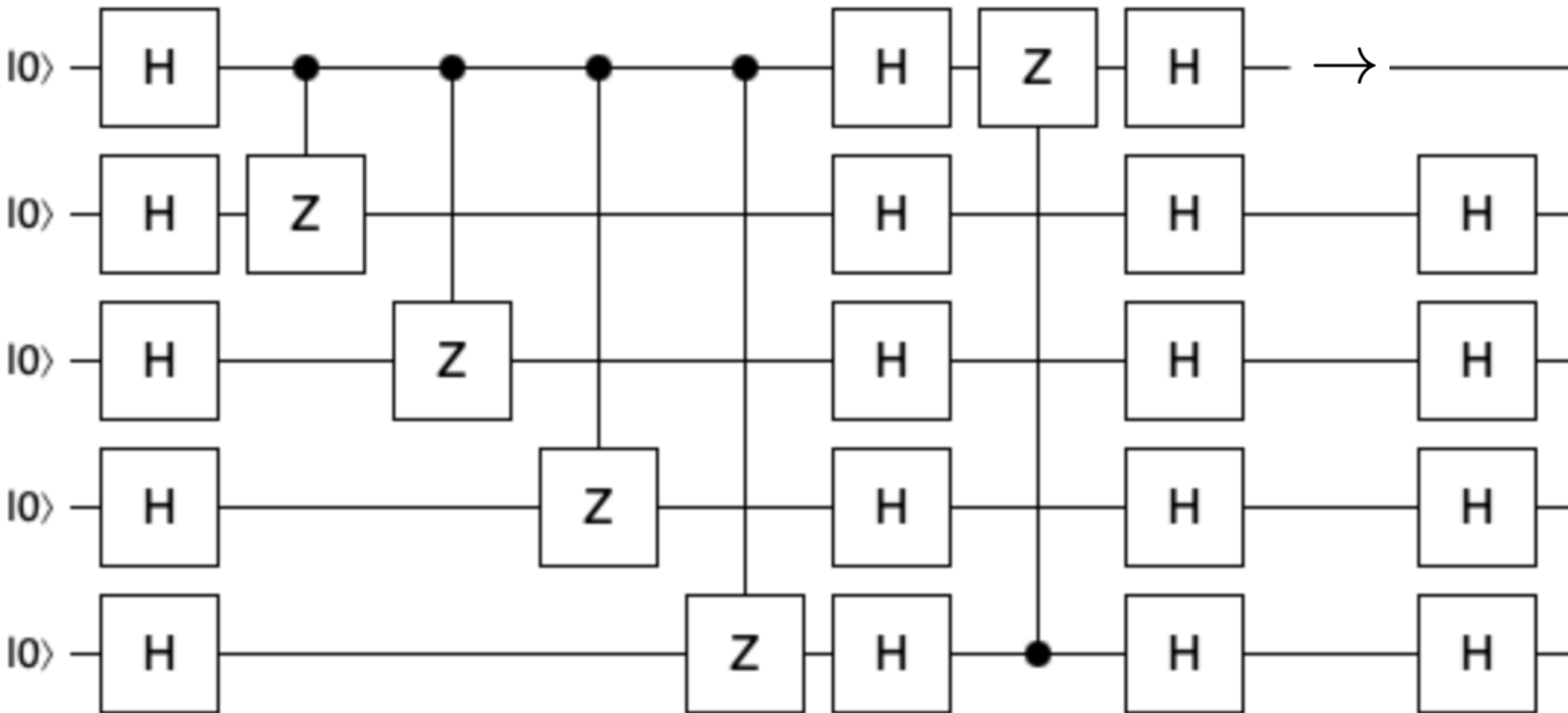
Quantum circuit, from theory to hardware



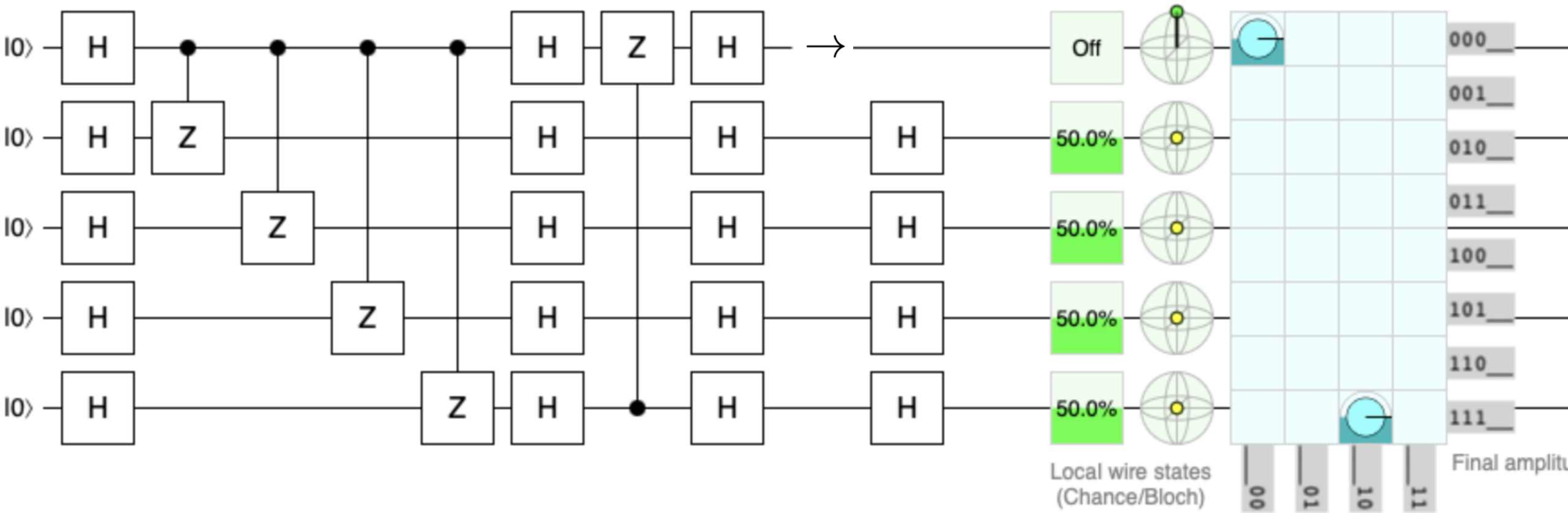
Quantum circuit, from theory to hardware



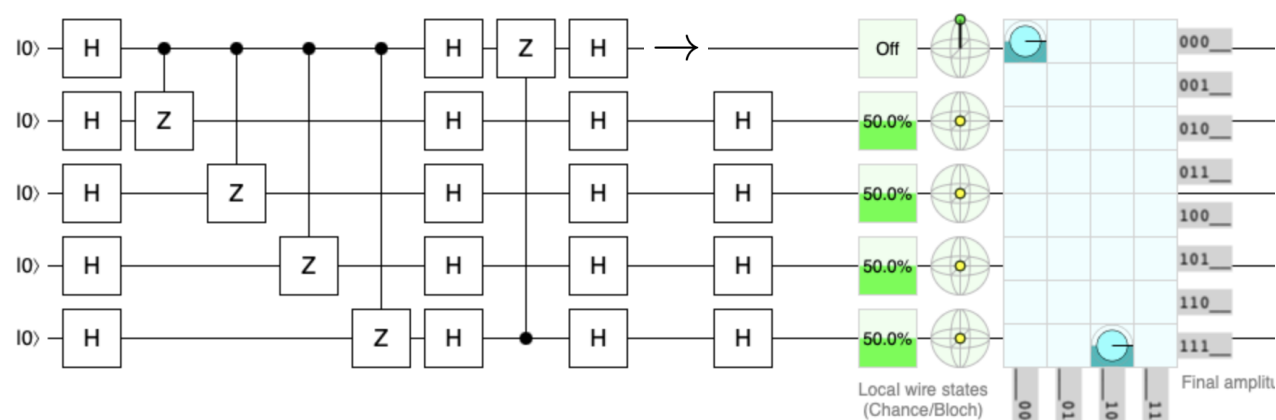
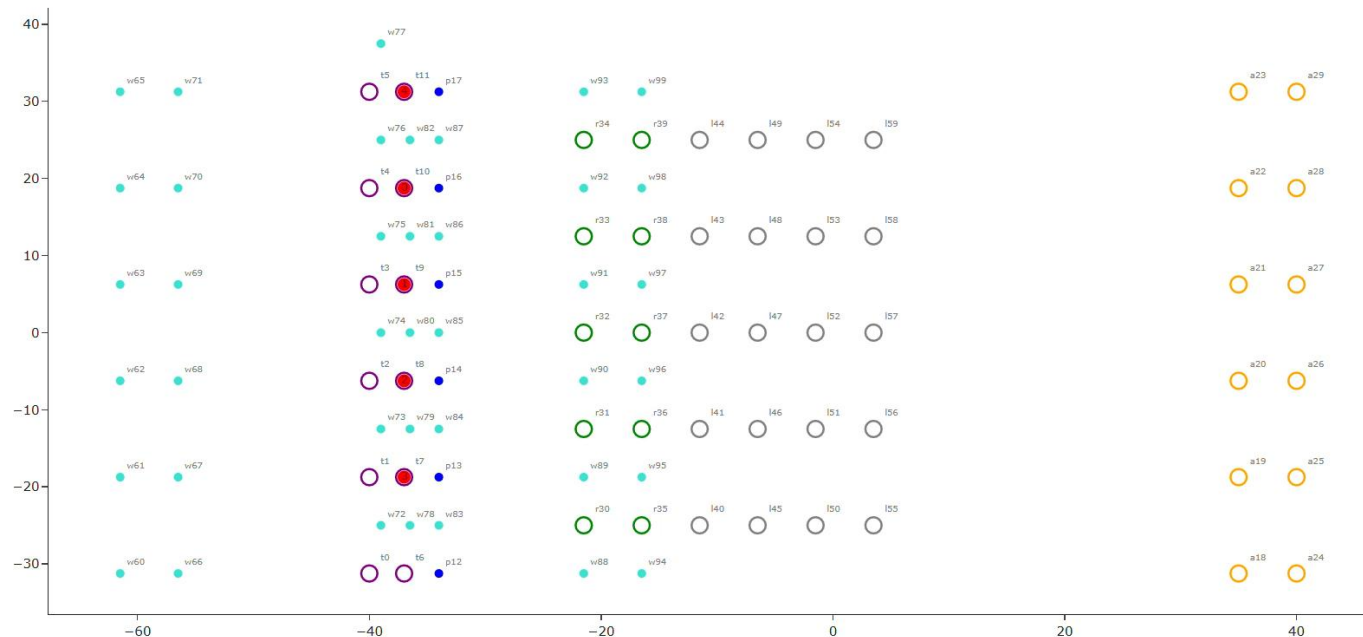
Quantum circuit, from theory to hardware



Quantum circuit, from theory to hardware



Atom Movement to Prepare $|00\rangle_L$

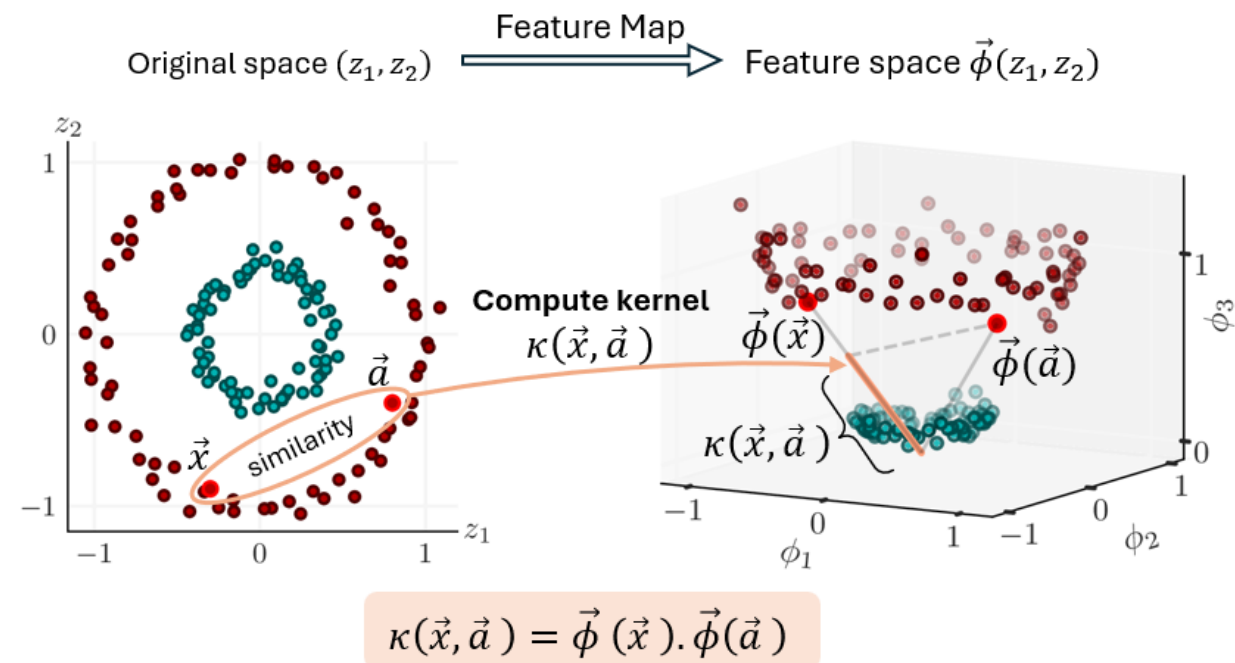


| Our Kernel Circuit



Kernel

- Mapping to higher dimensional space is useful.
- Kernel κ : mathematical object which measures similarities between two points.
- Key concepts:
 1. Similarity between points = inner product in a higher dimension space (feature space)
 2. The kernel is a shortcut to evaluate the similarity without requiring to map the data explicitly into the feature space

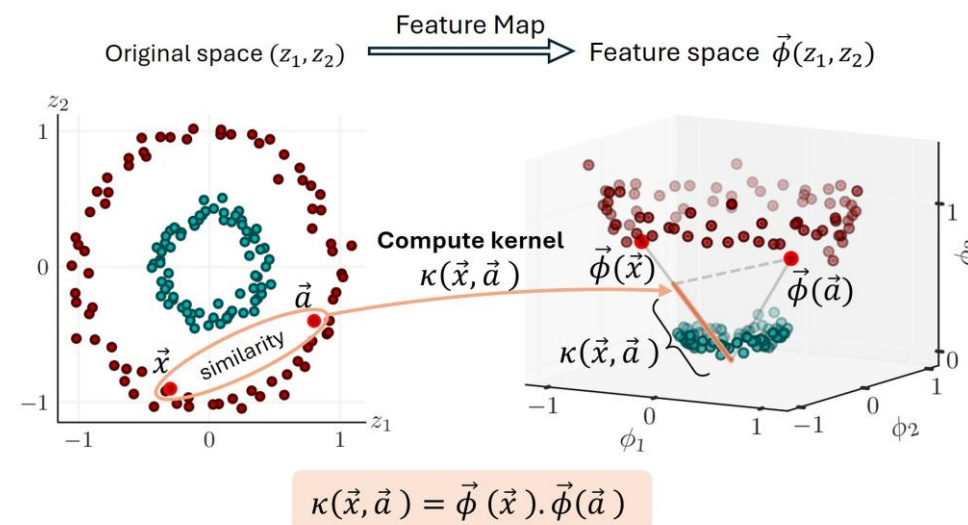
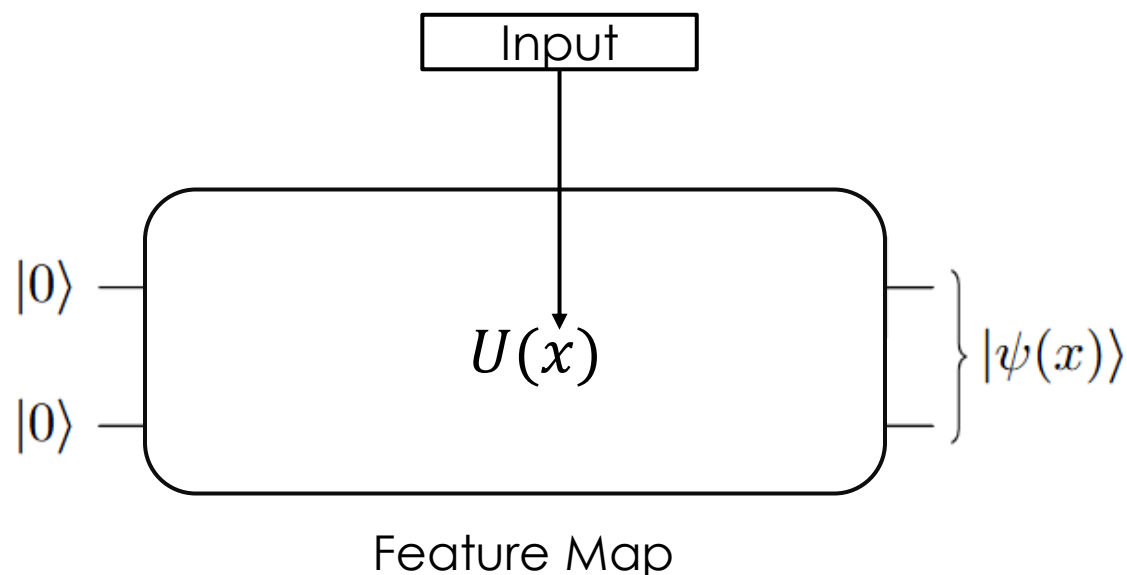


Typical applications:

- Machine learning (classification, etc...)
- Image processing, handwriting analysis...

Quantum Kernel Feature Map

The large Hilbert space accessible to quantum circuits makes it a natural candidate as a feature space to map classical inputs.

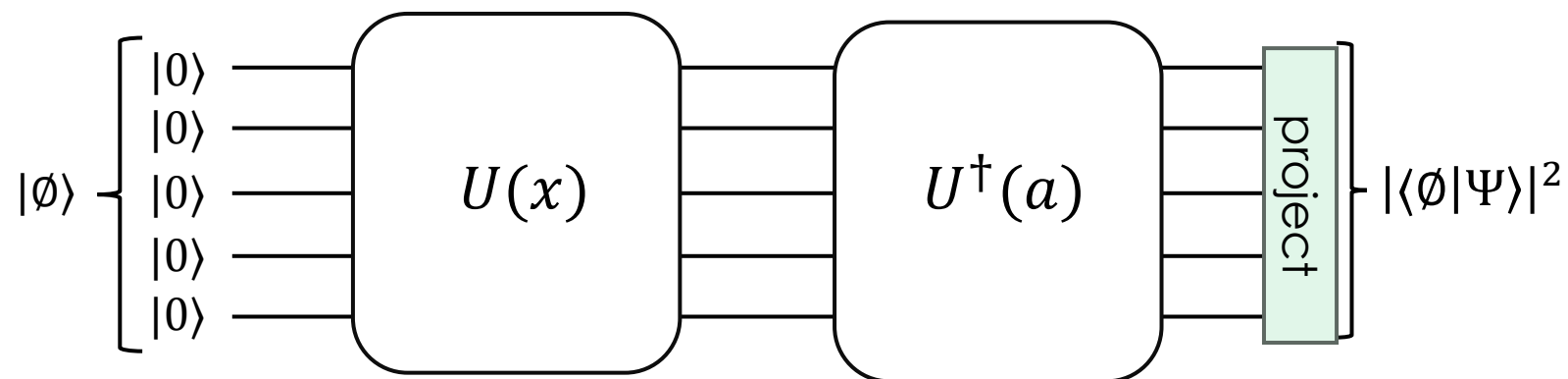


How do we perform the inner product?

Inner Product for the Quantum Kernel

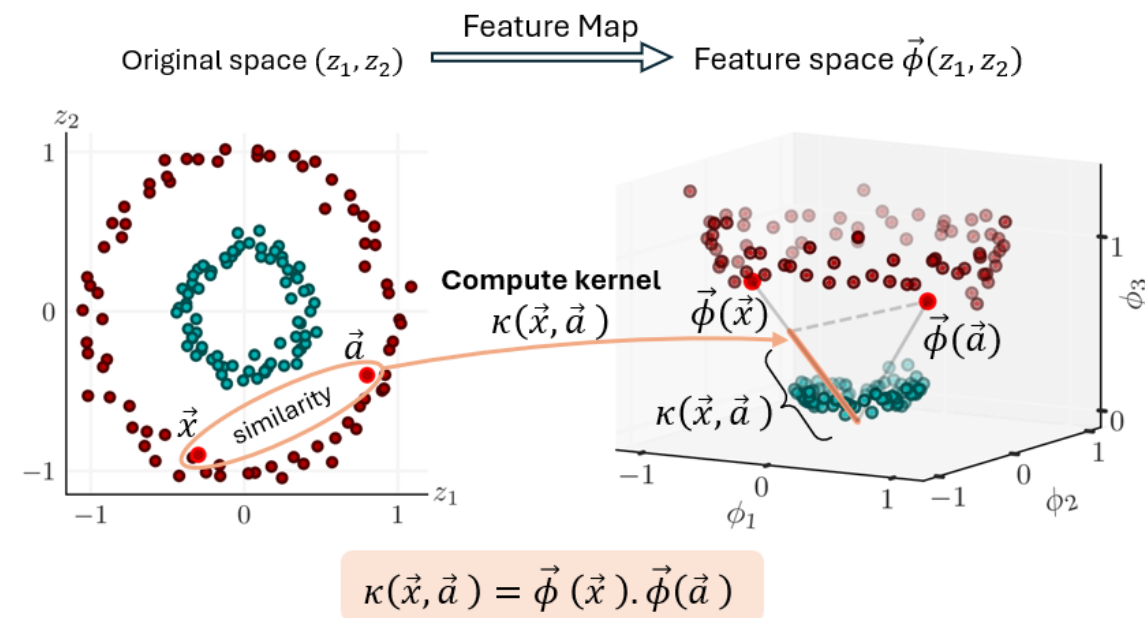
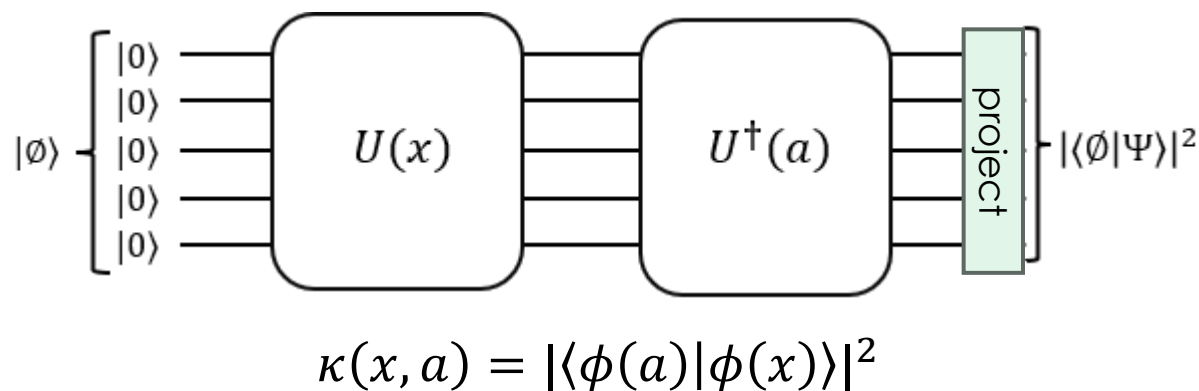
Defined by a unitary U such that:
$$\begin{cases} U(a)|\emptyset\rangle = |\phi(a)\rangle \\ U(x)|\emptyset\rangle = |\phi(x)\rangle \end{cases} \quad \kappa(x, a) = |\langle\phi(a)|\phi(x)\rangle|^2$$

How? Measure the return probability into the initial state of the following circuit:



$$|\langle\emptyset|\Psi\rangle|^2 = |\langle\emptyset|U(x)U^\dagger(a)|\emptyset\rangle|^2 = |\langle\phi(a)|\phi(x)\rangle|^2$$

Kernel to Quantum Kernel



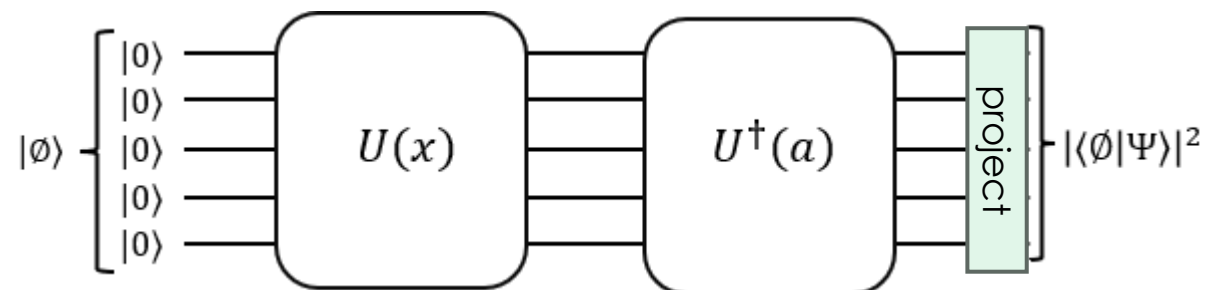
Note:

It has been demonstrated that quantum kernels can outperform their classical counterparts in some cases [1-3]. Solving differential equations is *not* one of them.

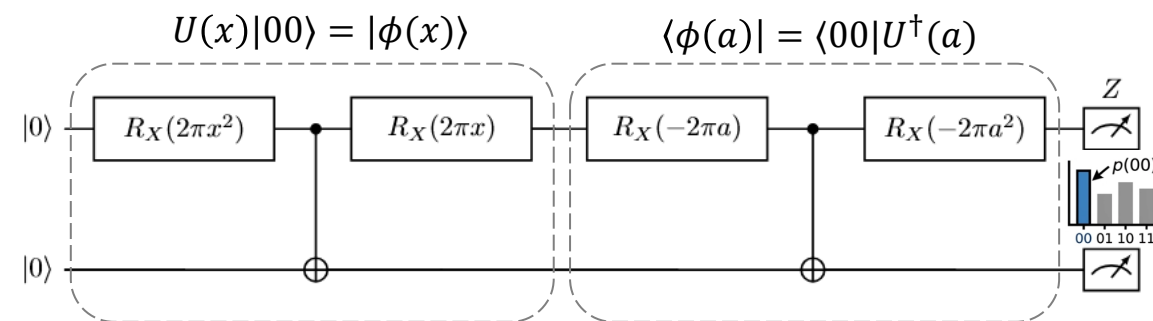
- [1] V. Havlicek, et. al (2019)
- [2] Y. Liu, et. al (2021)
- [3] E. Gil-Fuster, et. al (2025)

Our implementation

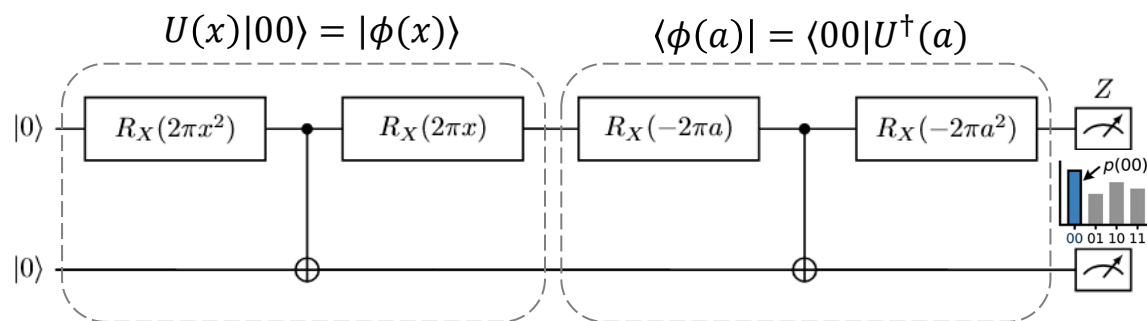
$$\kappa(x, a) = |\langle \phi(a) | \phi(x) \rangle|^2$$



Our implementation:



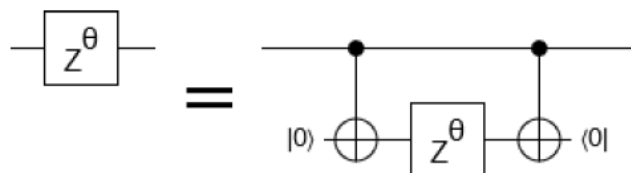
Our implementation: logical qubits



Requirements:

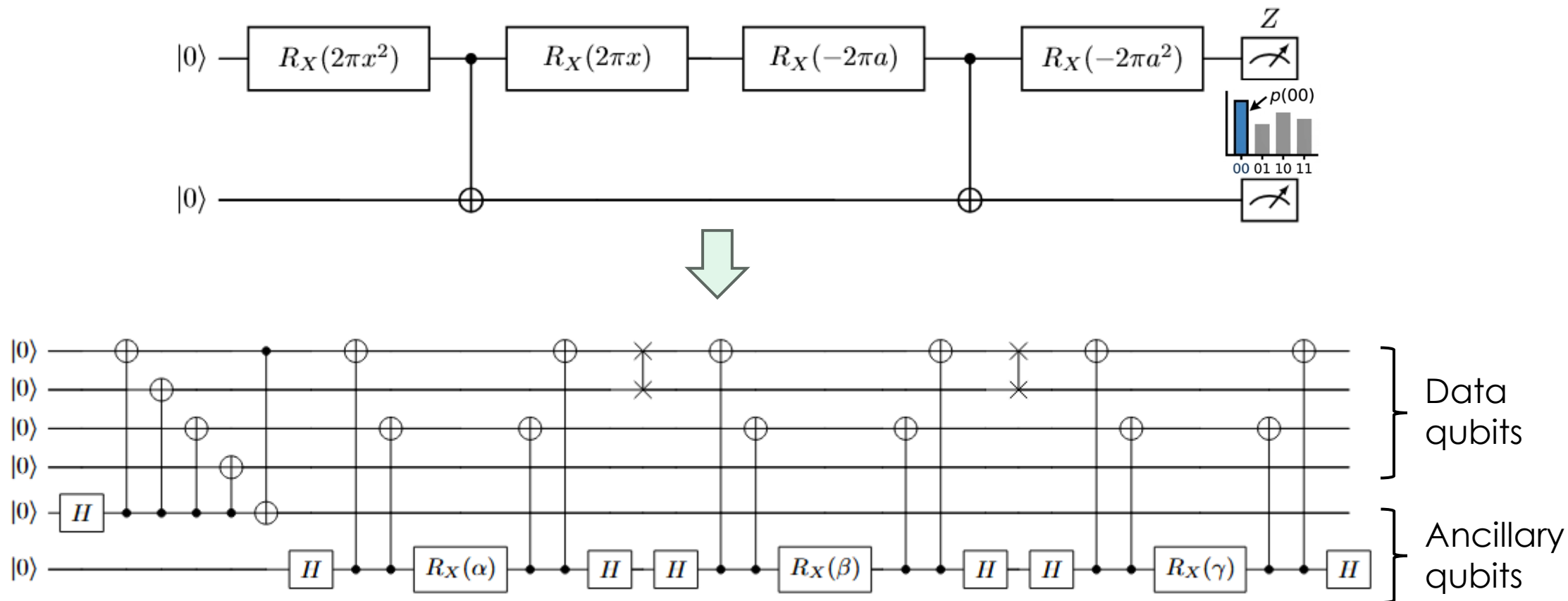
- Prepare the $|00\rangle_L$ state: $|00\rangle_L \rightarrow |0000\rangle + |1111\rangle$
- Apply C-NOT gates: swap data qubits
- Apply the R_X **non-Clifford** gates: the hard part
→ use a partially-FT method proxy phasing

Proxy phasing



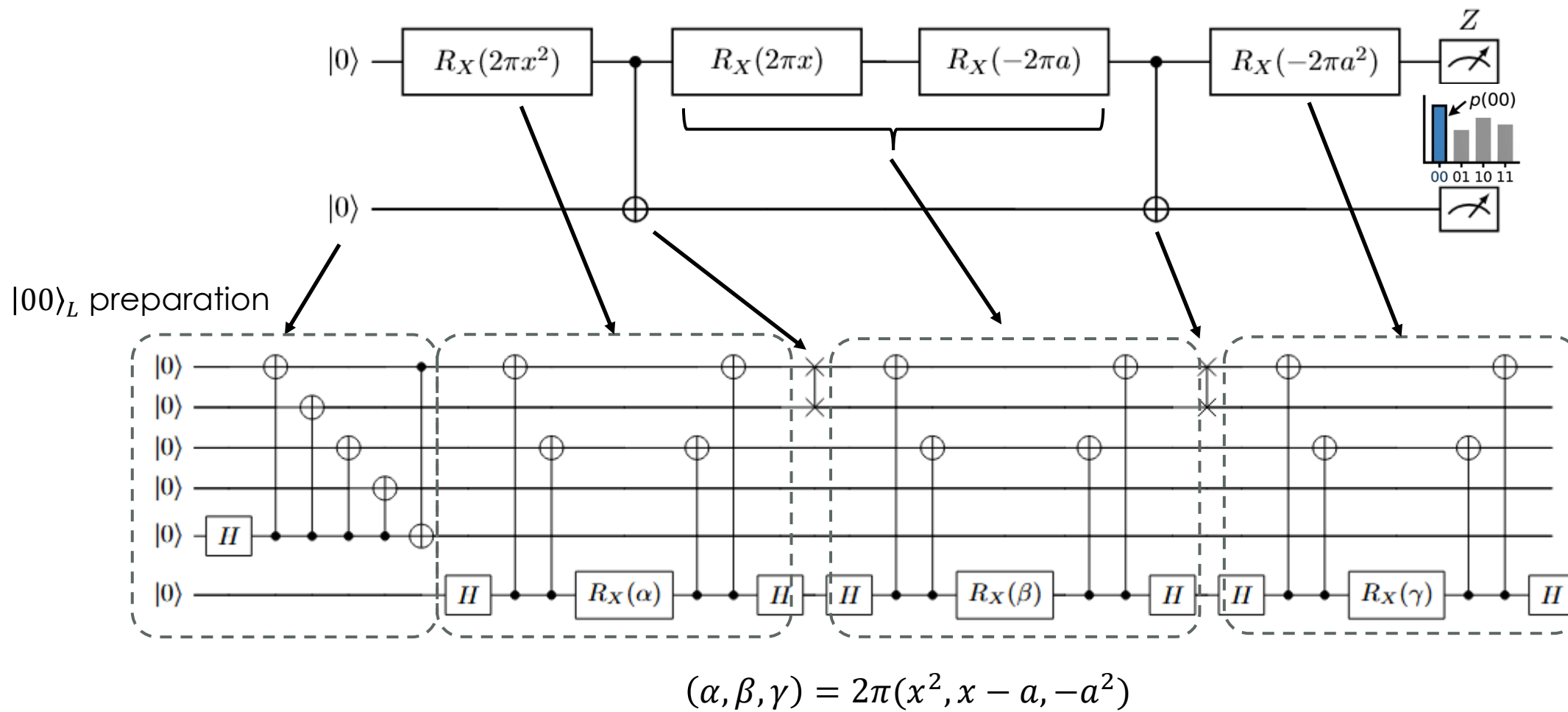
Operation	Unencoded	FT Encoded
$X(q0)$		
$Z(q0)$		
$X(q1)$		
$Z(q1)$		
$CZ(q0, q1)$		
$CX(q0, q1)$		
$CX(q1, q0)$		
$HH(q0, q1)$		

Our implementation: logical qubits

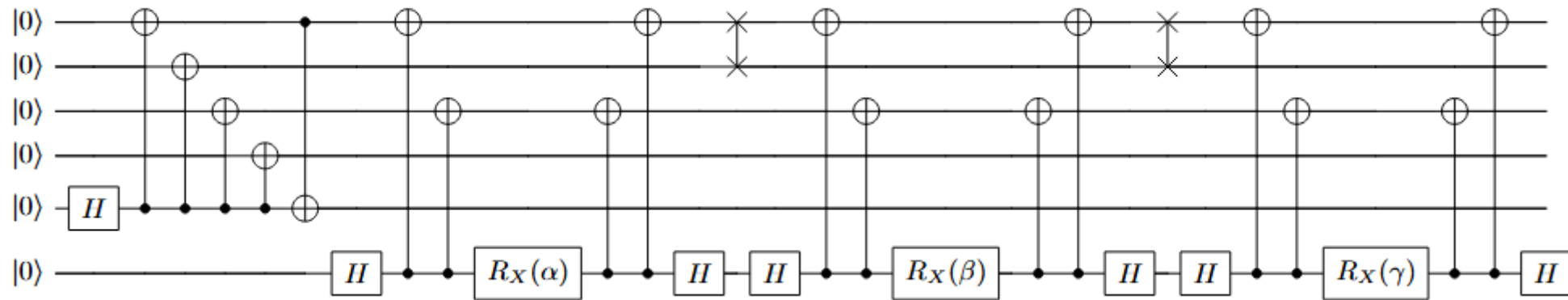


$$(\alpha, \beta, \gamma) = 2\pi(x^2, x - a, -a^2)$$

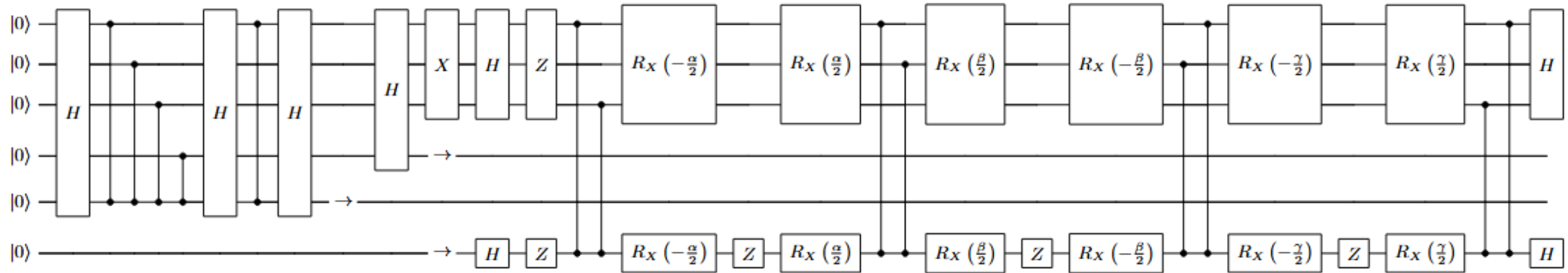
Our implementation: logical qubits



Our implementation: logical qubits



compilation (account for global beam)

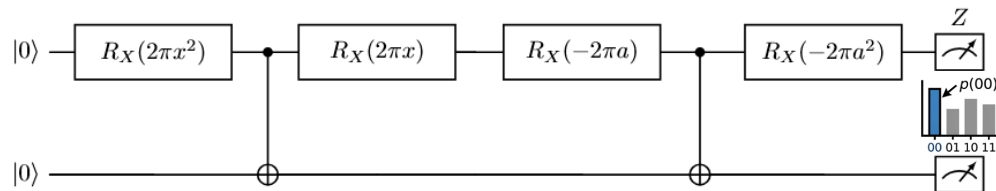


... which is still missing the atom movements and the dynamical decoupling

Kernel Results

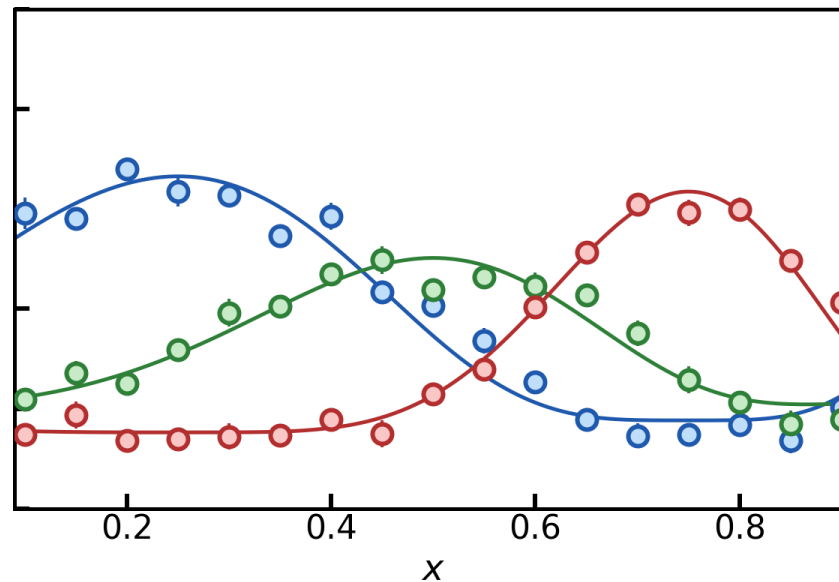
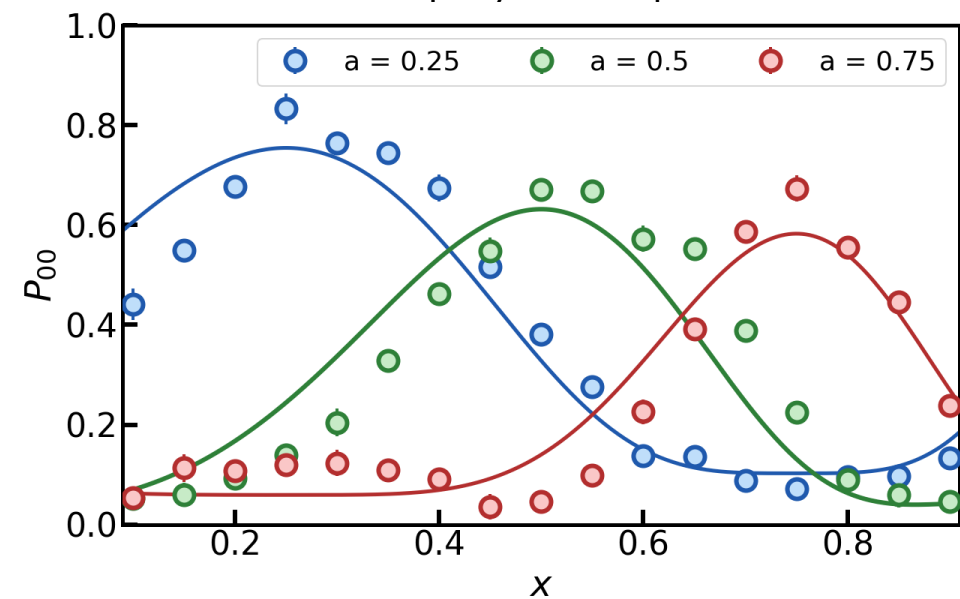


Results



Quantum kernel computed with physical qubits

Quantum kernel computed with logical qubits



Solid line: ideal kernel, after rescaling and shifting*

Observations:

- Contrast is lower for logical qubits, but...
- ... perhaps the shape is better?

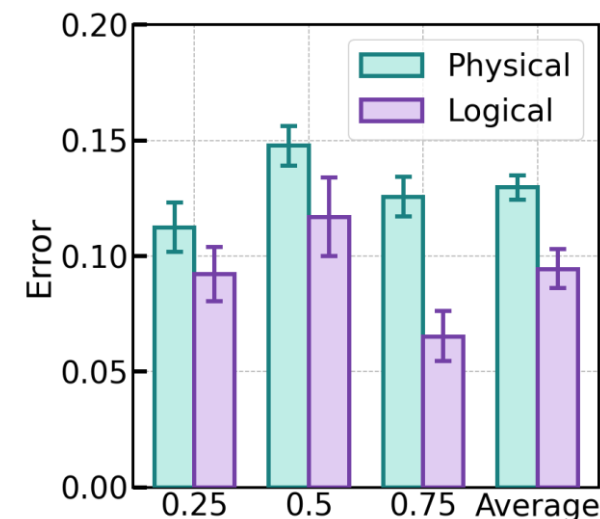
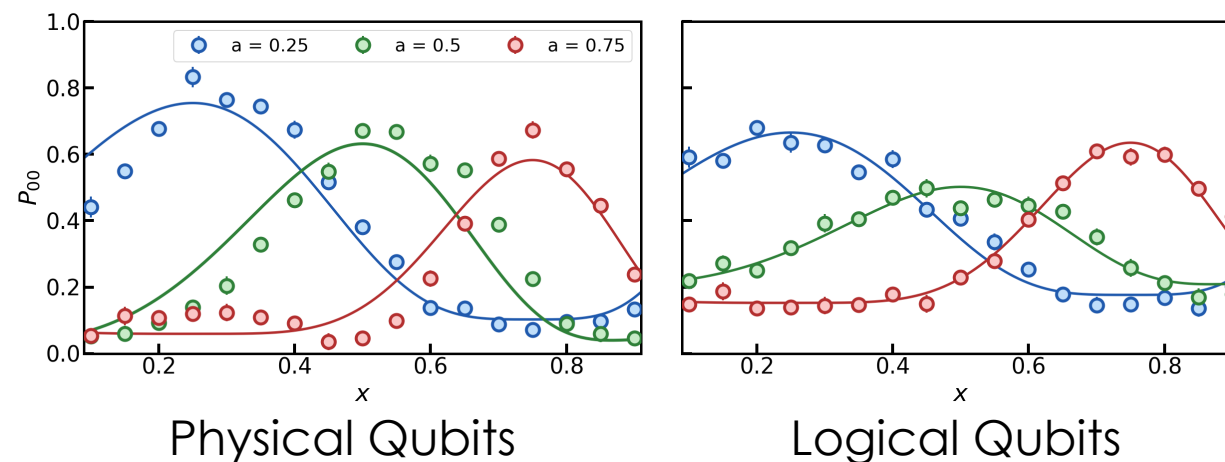
Results

- Recall

$$f_{\text{tr}}(x) = \sum w_i \kappa(x, a_i) + c$$

Classical Optimization Parameters

1. Rescaling of the kernel does not matter
 2. Constant shifts of the kernel does not matter
- We calculate the RMSE after rescaling and shifting, for quantitative comparison.

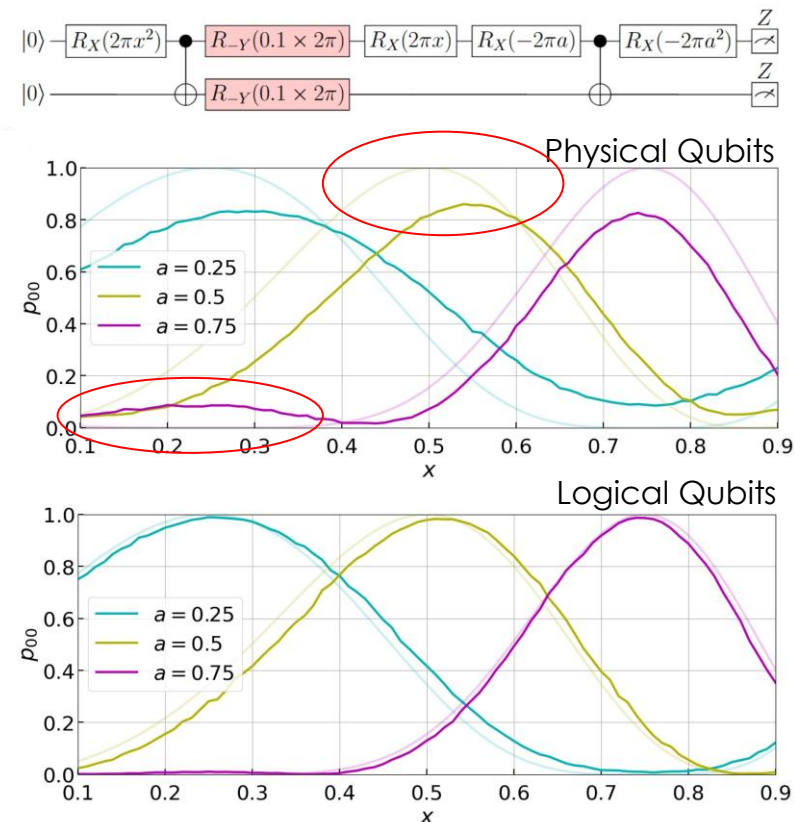
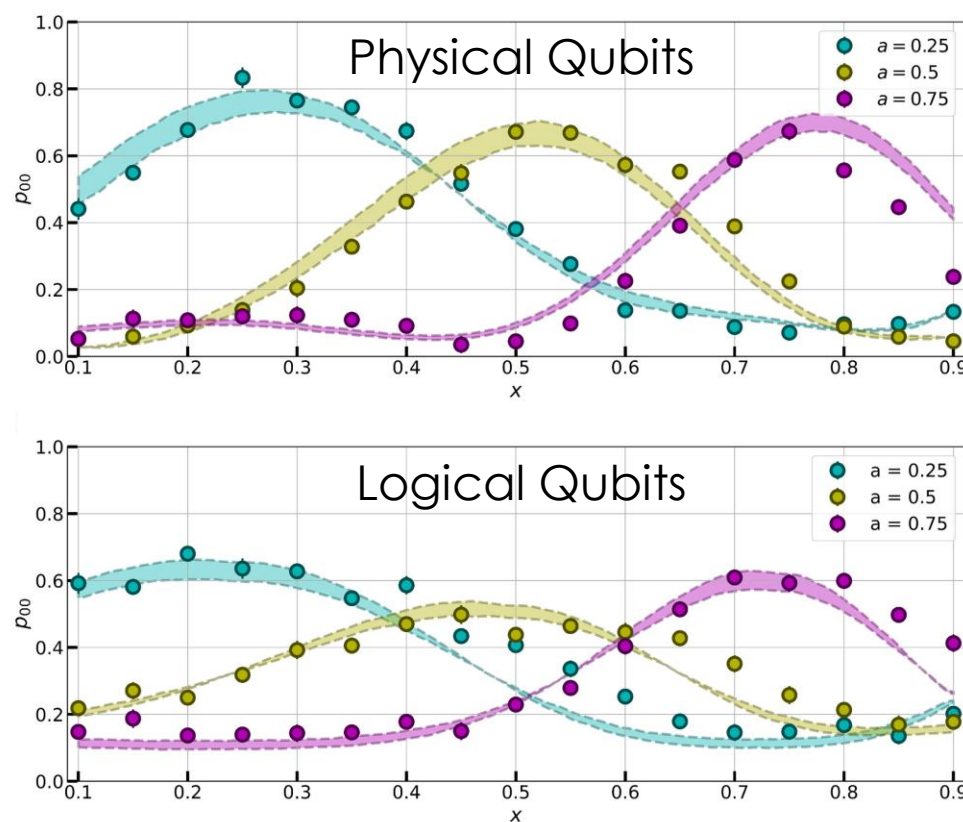


Logical is better!

Simulation Results

Was this chance? Unlikely, we reproduced this behavior in simulation.

- All the hardware operations (dynamical decoupling pulses, coherent moves,...)
- Ideal unitary + Pauli error channels + Atom-loss channel.
- Pauli errors and coherent errors mostly responsible for distortion.

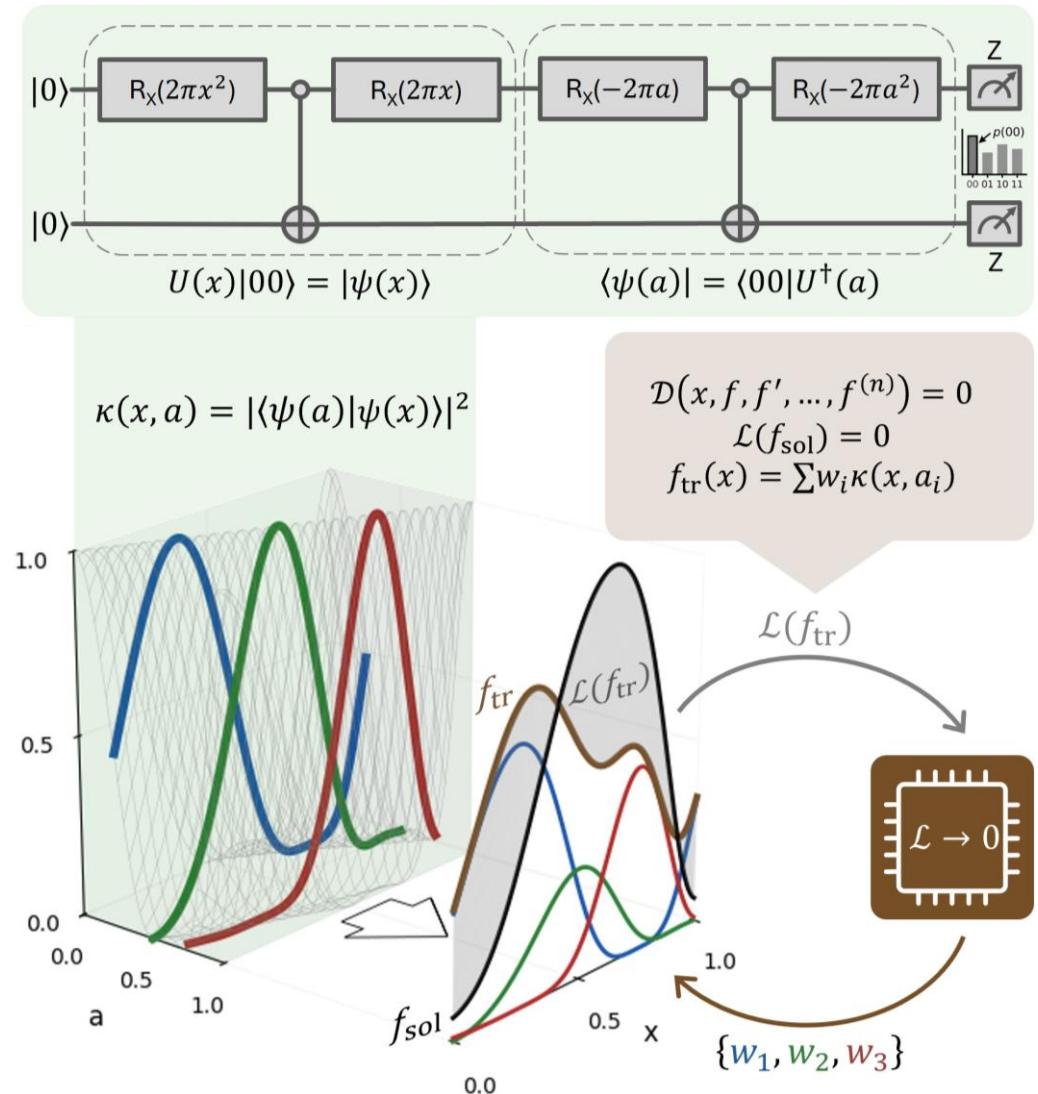


| Solving Differential Equations



Solving Differential Equations

- From RMSE of kernel, we expect the kernel to perform better, but does it?
- Recall key points of the method:
 - The kernel provides a basis for a trial function $f_{\text{tr}}(x) = \sum w_i \kappa(x, a_i) + c$
 - Minimize $\mathcal{L}(f_{\text{tr}}) \rightarrow 0$ via gradient descent classically



Solving Differential Equations

Differential Equation

$$\frac{1}{5} \frac{df(x)}{dx} + f(x) - g(x) = 0; f(1/2) = 2$$

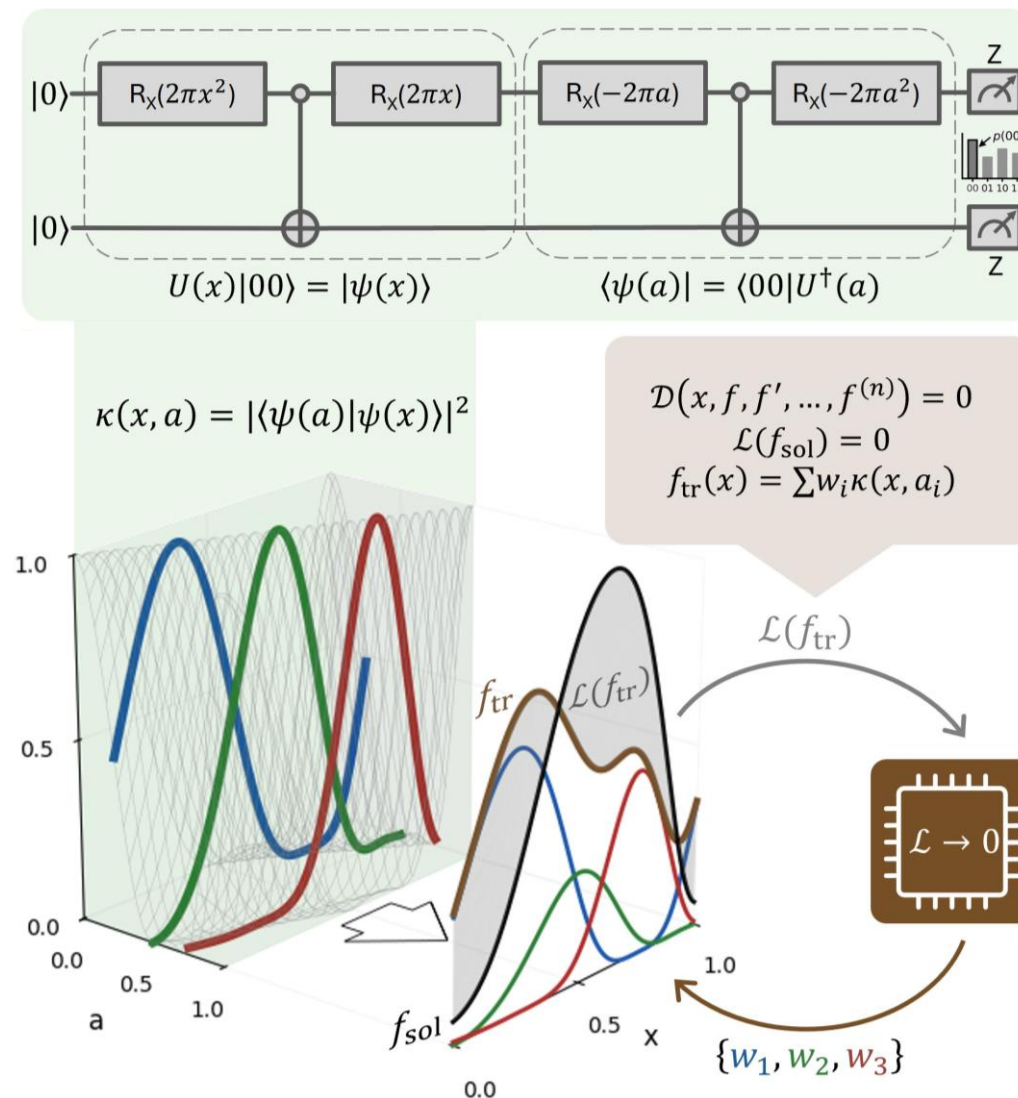
Trial Function

$$f_{\text{tr}}(x) = \sum w_i \kappa(x, a_i) + c$$

Loss function

$$\mathcal{L}(\{w_i\}, c) = \sum_{j=1}^N \frac{1}{N} \left(\frac{1}{5} f'_{\text{tr}} + f_{\text{tr}} - g \right)^2 \Big|_{x=x_j} + 10 (f_{\text{tr}} - 2)^2 \Big|_{x=1/2}$$

Optimized Classically



Benchmarking

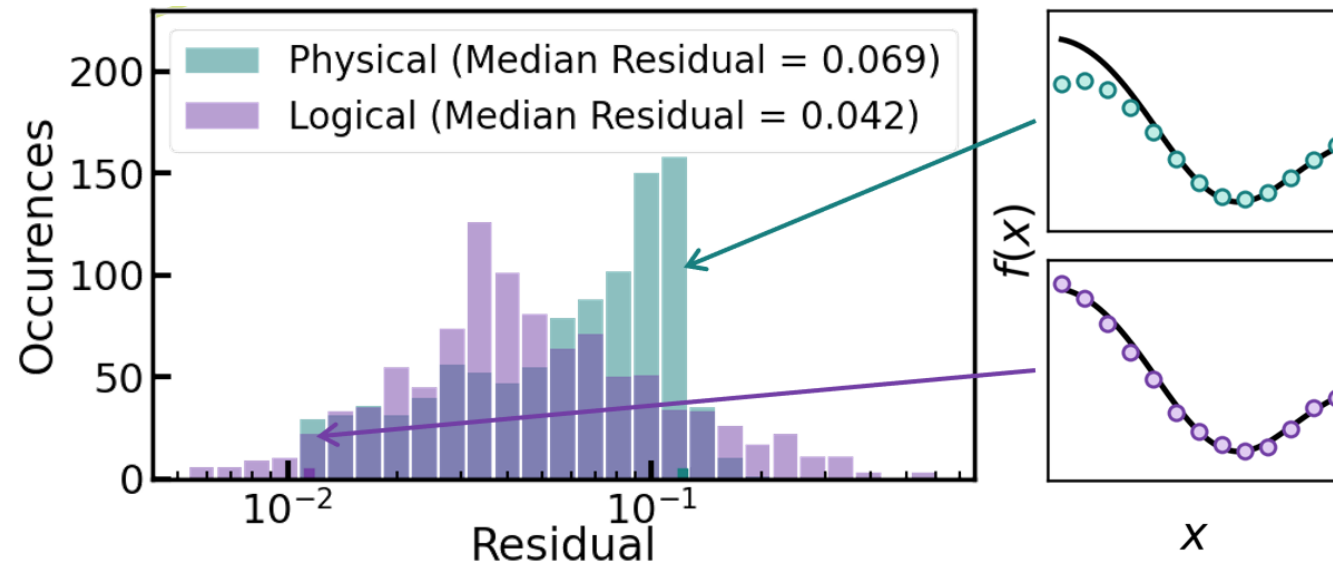
- We solve

$$\frac{1}{5} \frac{df(x)}{dx} + f(x) - g(x) = 0; f(1/2) = 2$$

1000 times with random $g(x)$.

Note: $g(x)$ is constructed such that the ideal circuit could perfectly solve this.

- We compare the obtained solution to the theoretical solution.



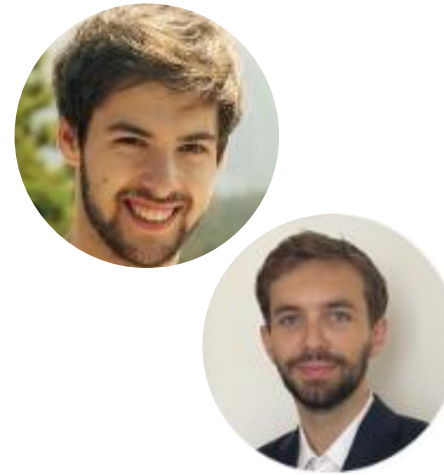
- This was co-design. Hardware improvements happened in parallel, closely driven by the selected application.
- Behind the scenes: In addition to the hardware a lot of tools, like the grammar to specify the algorithm and tools simulate output, were also developed. Many alternate implementation strategies were attempted.
- We are using one of the smallest error detecting codes and is not below threshold. Still, there seems to be some degree of error mitigation in an end-to-end application!

Thank you!

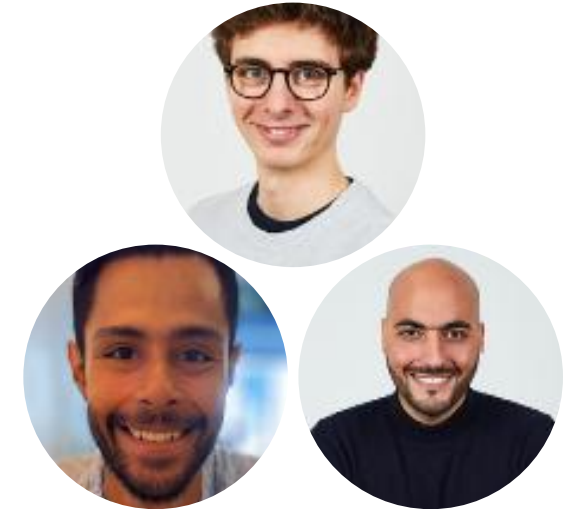
Core hardware team



QEC / theory



Simulations



... and many more.



arXiv:2605.21276

**Benchmarking a machine-learning differential equations solver
on a neutral-atom logical processor**